

Guida Completa a JavaScript: Da Principiante ad Avanzato

Scopri il mondo della programmazione JavaScript con la nostra guida dettagliata che ti accompagnerà dal livello principiante fino alle tecniche più avanzate, con esempi pratici ed esercitazioni guidate per un apprendimento efficace.

Inizia il Corso

Vedi il Programma

Cos'è JavaScript e Dove si Usa

JavaScript è un linguaggio di programmazione interpretato che funziona sia lato client (nel browser) che lato server (con Node.js). È il linguaggio principale per rendere interattive le pagine web e creare applicazioni dinamiche.

Basato sullo standard ECMAScript, JavaScript garantisce compatibilità tra diversi browser e piattaforme, rendendolo uno strumento versatile e potente per gli sviluppatori moderni.

Principali ambiti di utilizzo:

- Sviluppo di siti web dinamici e interattivi
- Creazione di applicazioni mobile ibride
- Programmazione backend con Node.js
- Sviluppo di applicazioni desktop con Electron



JavaScript è oggi onnipresente: dal browser al server, dalle applicazioni mobile ai dispositivi IoT, rappresentando una competenza fondamentale per qualsiasi sviluppatore moderno.

Setup dell'Ambiente di Sviluppo

Installazione di Node.js

Node.js è un runtime JavaScript che permette di eseguire codice JavaScript al di fuori del browser. Per installarlo:

1. Visita nodejs.org e scarica la versione LTS
2. Segui la procedura guidata di installazione
3. Verifica l'installazione con `node -v` nel terminale

Editor di Testo

Un buon editor di codice è essenziale. Consigliamo:

- Visual Studio Code: leggero e con estensioni per JavaScript
- WebStorm: IDE completo, ideale per progetti più complessi
- Atom o Sublime Text: alternative valide e personalizzabili

Strumenti di Debug

I browser moderni offrono potenti strumenti di sviluppo:

- Chrome DevTools: premi F12 o click destro → Ispeziona
- Firefox Developer Tools: similmente con F12
- Utilizza la console per testare rapidamente il codice

Puoi eseguire JavaScript direttamente nel browser (aprendo la console) o nel terminale con Node.js. Per i principianti, consigliamo di iniziare con semplici script in file HTML per visualizzare immediatamente i risultati.

Sintassi Base di JavaScript

La sintassi di JavaScript definisce le regole fondamentali per scrivere codice valido. Comprendere queste basi è essenziale per evitare errori e scrivere programmi funzionanti.

Commenti nel Codice

```
// Commento su una singola riga

/* Commento
   su più
   righe */
```

Regole per Identificatori

Gli identificatori (nomi di variabili, funzioni, ecc.) devono seguire queste regole:

- Possono contenere lettere, numeri, underscore (_) e dollaro ($)
- Non possono iniziare con un numero
- Sono case-sensitive (myVar ≠ MyVar)
- Non possono essere parole riservate (come if, for, function)

Convenzioni di Scrittura

JavaScript è case-sensitive e, sebbene il punto e virgola alla fine delle istruzioni sia spesso facoltativo, è una buona pratica includerlo sempre per evitare ambiguità. L'indentazione del codice è importante per la leggibilità, specialmente nei blocchi annidati.

Aggiungere JavaScript a una Pagina Web

Tre Modi per Includere JavaScript

JavaScript Inline

Direttamente nell'attributo di un elemento HTML.

```
<button onclick="alert('Ciao!')">Clicca</button>
```

Adatto solo per codice brevissimo, sconsigliato per la maggior parte dei casi.

JavaScript Interno

All'interno del documento HTML usando il tag script.

```
<script>
  function saluta() {
    alert('Ciao mondo!');
  }
</script>
```

Utile per script specifici della pagina.

JavaScript Esterno

In file separati .js collegati all'HTML.

```
<script src="mioScript.js"></script>
```

Approccio raccomandato: migliora manutenibilità e cache.

Attributi Speciali

Gli attributi `async` e `defer` controllano il caricamento degli script:

```
<script src="analytics.js" async></script>
<script src="main.js" defer></script>
```

- **async**: Lo script viene eseguito in modo asincrono appena disponibile
- **defer**: Lo script viene eseguito solo dopo il parsing completo dell'HTML

Il posizionamento dello script influisce sulle performance:

- Alla fine del body: caricamento più veloce della pagina
- Nell'head con defer: compromesso ottimale



Variabili e Costanti

Dichiarazione delle Variabili

JavaScript offre tre modi per dichiarare variabili, ognuno con comportamenti diversi:

var

La dichiarazione tradizionale, con scope di funzione o globale.

```
var nome = 'Mario';
var età = 30;
var occupato = true;
```

Problemi: hoisting, scope di funzione (non di blocco), possibilità di ridichiarazione.

let

Introdotto in ES6, con scope di blocco e migliore comportamento.

```
let nome = 'Luigi';
let punteggio = 100;
```

Vantaggi: scope di blocco, no ridichiarazione nello stesso scope, hoisting senza inizializzazione.

const

Per valori che non devono cambiare dopo l'assegnazione iniziale.

```
const PI = 3.14159;
const GIORNI_SETTIMANA = 7;
```

Nota: per oggetti e array, previene la riassegnazione ma non la modifica interna.

Scope e Hoisting

Lo **scope** definisce dove una variabile è accessibile nel codice:

- **Scope globale**: variabili dichiarate fuori da qualsiasi funzione o blocco
- **Scope di funzione**: variabili dichiarate all'interno di una funzione
- **Scope di blocco**: variabili let/const dichiarate all'interno di un blocco {}

L'**hoisting** è il comportamento di JavaScript di "sollevare" le dichiarazioni all'inizio del loro scope:

- Le variabili var vengono sollevate e inizializzate a undefined
- Le variabili let/const vengono sollevate ma non inizializzate (Temporal Dead Zone)
- Le dichiarazioni di funzione vengono sollevate completamente

Operatori e Espressioni

Operatori Aritmetici

```
+    // Addizione: 5 + 2 = 7
-    // Sottrazione: 5 - 2 = 3
*    // Moltiplicazione: 5 * 2 = 10
/    // Divisione: 5 / 2 = 2.5
%    // Modulo (resto): 5 % 2 = 1
**   // Esponente: 5 ** 2 = 25
++   // Incremento: ++x o x++
--   // Decremento: --x o x--
```

Operatori di Confronto

```
==   // Uguale (valore): 5 == "5" è true
===  // Strettamente uguale (valore e tipo): 5 === "5" è false
!=   // Diverso (valore): 5 != "6" è true
!==  // Strettamente diverso: 5 !== "5" è true
>    // Maggiore di: 5 > 3 è true
<    // Minore di: 5 < 3 è false
>=   // Maggiore o uguale: 5 >= 5 è true
<=   // Minore o uguale: 5 <= 5 è true
```

Operatori di Assegnazione

```
=    // Assegnazione: x = 5
+=   // x += 3 equivale a x = x + 3
-=   // x -= 3 equivale a x = x - 3
*=   // x *= 3 equivale a x = x * 3
/=   // x /= 3 equivale a x = x / 3
%=   // x %= 3 equivale a x = x % 3
```

Operatori Logici

```
&&   // AND logico: true && false è false
||   // OR logico: true || false è true
!    // NOT logico: !true è false
??   // Nullish coalescing: null ?? "default" è "default"
```

Operatori Speciali

Operatore Ternario

Un modo conciso per scrivere espressioni condizionali:

```
condizione ? valoreSeVero : valoreSefalso

// Esempio
let stato = età >= 18 ? "adulto" : "minorenne";
```

Operatore Spread

Espande array o oggetti nei loro elementi:

```
// Per array
let numeri = [1, 2, 3];
let nuoviNumeri = [...numeri, 4, 5];  // [1, 2, 3, 4, 5]

// Per oggetti
let persona = {nome: "Mario", età: 30};
let dettagli = {...persona, lavoro: "sviluppatore"};
```

Precedenza degli Operatori

Gli operatori seguono una gerarchia di precedenza:

1. Parentesi tonde ()
2. Incremento/decremento, negazione unaria
3. Moltiplicazione, divisione, modulo
4. Addizione, sottrazione
5. Confronti (>, <, >=, <=)
6. Uguaglianza (==, ===, !=, !==)
7. AND logico (&&)
8. OR logico (||)
9. Assegnazione (=, +=, -=, ecc.)

Stringhe e Manipolazione Testi

Creazione e Concatenazione

```
// Creazione stringhe
let nome = 'Mario';
let cognome = "Rossi";

// Concatenazione
let nomeCompleto = nome + ' ' + cognome;  // "Mario Rossi"

// Template literals (ES6)
let presentazione = `Ciao, mi chiamo ${nome} ${cognome} e ho ${30} anni.`;
```

Metodi Principali delle Stringhe

1

Accesso e Ricerca

```
// Lunghezza
let lunghezza = nome.length;  // 5

// Accesso ai caratteri
let primaLettera = nome[0];  // 'M'
let ultimaLettera =
nome[nome.length-1];  // 'o'

// Ricerca
let posizione = nome.indexOf('ar');  //
1
let include = nome.includes('ar');  //
true
let iniziaCon = nome.startsWith('Ma');
// true
let finisceCon = nome.endsWith('io');
// true
```

2

Estrazione e Manipolazione

```
// Estrazione
let parte = nome.substring(1, 3);  // 'ar'
let sottostringa = nome.slice(1, 3);  //
'ar'
let estratto = nome.substr(1, 2);  // 'ar'

// Divisione
let parole = "ciao mondo".split(' ');  //
['ciao', 'mondo']
let lettere = nome.split('');  // ['M', 'a',
'r', 'i', 'o']

// Sostituzione
let nuovo = nome.replace('o', 'a');  //
'Maria'
let tutteSostituzioni =
nome.replaceAll('o', 'a');  // con ES2021
```

3

Formattazione

```
// Case
let maiuscolo = nome.toUpperCase();
// 'MARIO'
let minuscolo = nome.toLowerCase();
// 'mario'

// Spazi
let testo = "  ciao  ";
let pulito = testo.trim();  // 'ciao'
let pulitoDestra = testo.trimRight();  // '
ciao'
let pulitoSinistra = testo.trimLeft();  //
'ciao  '

// Riempimento
let codice = "42";
let riempito = codice.padStart(5, '0');
// '00042'
let riempitoDopo = codice.padEnd(5,
'0');  // '42000'
```

Esercitazione: Elaborazione Testo

```
// Dato un input utente, creare una funzione che:
// 1. Rimuova spazi iniziali e finali
// 2. Converta la prima lettera in maiuscolo
// 3. Nasconda i numeri di telefono con asterischi

function elaboraTesto(input) {
 let pulito = input.trim();
 let primaMaiuscola = pulito.charAt(0).toUpperCase() + pulito.slice(1);
 // Nasconde numeri telefonici (formato italiano)
 let testoSicuro = primaMaiuscola.replace(/(\d{3})\d{4}(\d{3})/g, '$1****$2');
 return testoSicuro;
}

// Esempio
let risultato = elaboraTesto(" ciao, il mio numero è 3331234567 ");
console.log(risultato); // "Ciao, il mio numero è 333****567"
```

Numeri e Calcoli

Tipi di Numeri in JavaScript

JavaScript utilizza un unico tipo number per rappresentare sia numeri interi che decimali, basato sul formato IEEE 754 (floating point a 64 bit).

```
// Numeri interi
let intero = 42;

// Numeri decimali
let decimale = 3.14159;

// Notazione scientifica
let grande = 1e6;  // 1000000
let piccolo = 1e-6;  // 0.000001

// Numeri speciali
let infinito = Infinity;
let negativoInfinito = -Infinity;
let nonNumero = NaN;  // Not a Number

// BigInt (ES2020) per numeri interi di qualsiasi dimensione
let numerone = 9007199254740991n;
```

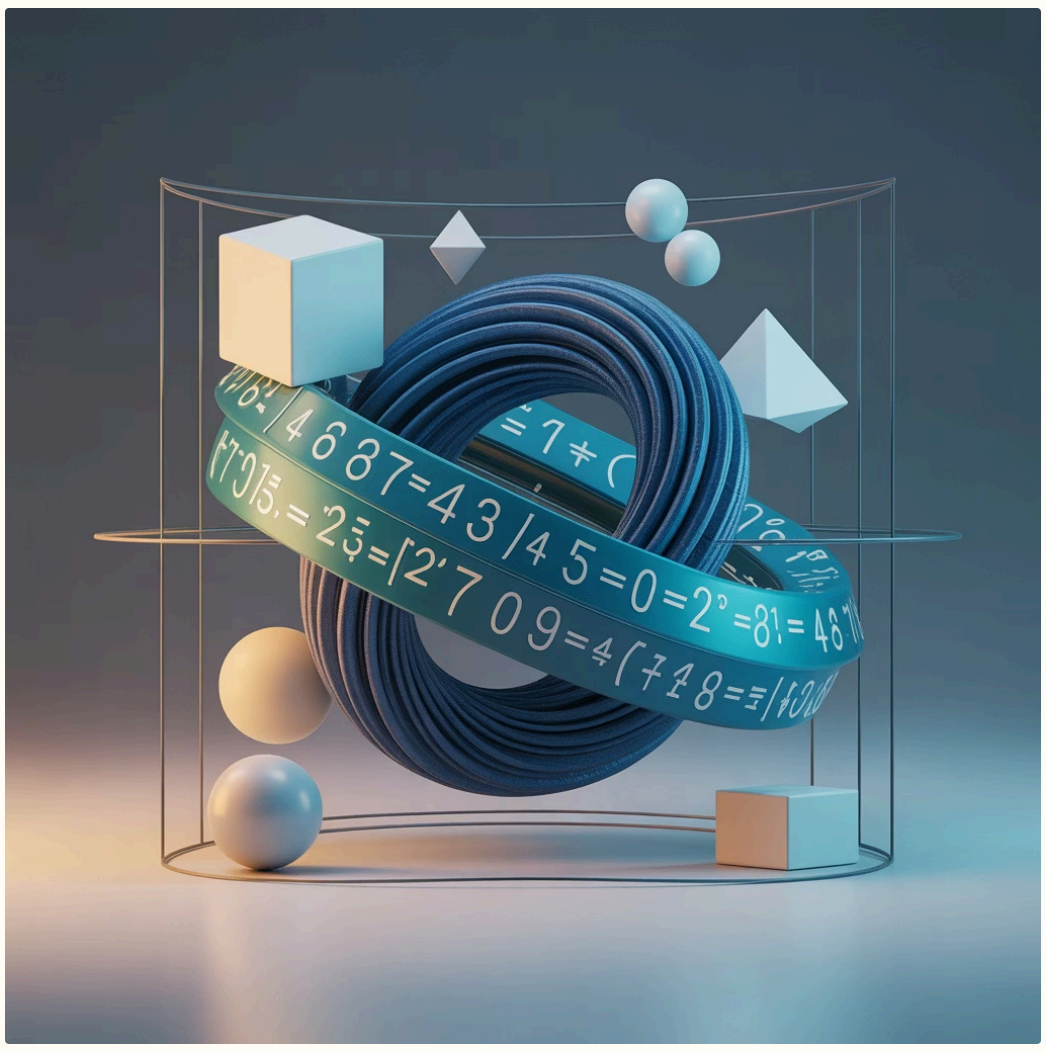
Verifiche su Numeri

```
// Controllo se è NaN
Number.isNaN(NaN);  // true
Number.isNaN("ciao");  // false

// Controllo se è numero finito
Number.isFinite(42);  // true
Number.isFinite(Infinity);  // false

// Controllo se è intero
Number.isInteger(42);  // true
Number.isInteger(42.0);  // true
Number.isInteger(42.5);  // false
```

Oggetto Math



L'oggetto Math fornisce proprietà e metodi per operazioni matematiche complesse:

```
// Costanti matematiche
Math.PI;  // 3.141592653589793
Math.E;   // 2.718281828459045

// Arrotondamenti
Math.round(3.7);  // 4
Math.floor(3.7);  // 3
Math.ceil(3.2);   // 4
Math.trunc(3.7);  // 3 (solo parte intera)

// Valore assoluto
Math.abs(-5);  // 5

// Potenze e radici
Math.pow(2, 3);  // 8 (2^3)
Math.sqrt(16);   // 4 (radice quadrata)
Math.cbrt(27);   // 3 (radice cubica)

// Min e Max
Math.min(2, 5, 1);  // 1
Math.max(2, 5, 1);  // 5

// Numeri casuali
Math.random();  // numero tra 0 (incluso) e 1 (escluso)
// Da 1 a 10:
Math.floor(Math.random() * 10) + 1;
```

Problemi con i Numeri Decimali

I calcoli con numeri decimali possono produrre risultati imprecisi a causa del formato floating point:

```
0.1 + 0.2;  // 0.30000000000000004
```

Soluzioni:

1. Arrotondare il risultato: `Math.round((0.1 + 0.2) * 100) / 100`
2. Usare toFixed: `(0.1 + 0.2).toFixed(2)` (restituisce una stringa)
3. Moltiplicare per potenze di 10, calcolare con interi, poi dividere

Array: Strutture e Metodi Fondamentali

Creazione e Accesso agli Array

```
// Creazione
let numeri = [1, 2, 3, 4, 5];
let misto = [1, 'due', true, {nome: 'Mario'}, [7, 8]];
let vuoto = new Array(3);  // array di 3 elementi vuoti

// Accesso
let primo = numeri[0];  // 1
let ultimo = numeri[numeri.length - 1];  // 5
let nonEsistente = numeri[10];  // undefined

// Proprietà length
let lunghezza = numeri.length;  // 5
```

Metodi di Base per Manipolare Array

1

Aggiungere e Rimuovere Elementi

```
// Fine array
numeri.push(6);  // aggiunge alla fine
numeri.pop();    // rimuove dall'ultima posizione

// Inizio array
numeri.unshift(0);  // aggiunge all'inizio
numeri.shift();     // rimuove dalla prima posizione

// Posizione specifica
numeri.splice(2, 1);  // rimuove 1 elemento dall'indice 2
numeri.splice(2, 0, 'nuovo');  // inserisce senza rimuovere
```

2

Ricerca e Selezione

```
// Verificare presenza
numeri.includes(3);  // true
numeri.indexOf(3);   // 2 (indice del primo 3)
numeri.lastIndexOf(3);  // ultimo indice di 3

// Trovare elementi
numeri.find(x => x > 3);  // primo elemento > 3
numeri.findIndex(x => x > 3);  // indice del primo > 3

// Slicing (estrazione porzioni)
let porzione = numeri.slice(1, 3);  // [2, 3]
```

3

Trasformazione Array

```
// Unire array
let uniti = numeri.concat([6, 7, 8]);

// Invertire
numeri.reverse();  // modifica l'array originale

// Ordinare
numeri.sort();  // ordine alfabetico di default
numeri.sort((a, b) => a - b);  // ordine numerico

// Trasformare in stringa
numeri.join('-');  // "1-2-3-4-5"
```

Iterazione sugli Array

```
// forEach: esegue una funzione per ogni elemento
numeri.forEach((elemento, indice) => {
  console.log(`Elemento ${indice}: ${elemento}`);
});

// map: crea un nuovo array trasformando ogni elemento
let quadrati = numeri.map(x => x * x); // [1, 4, 9, 16, 25]

// filter: crea un nuovo array con elementi che passano un test
let pari = numeri.filter(x => x % 2 === 0); // [2, 4]

// some/every: verificano se alcuni/tutti gli elementi soddisfano una condizione
let alcuniMaggioriDi3 = numeri.some(x => x > 3); // true
let tuttiMaggioriDi0 = numeri.every(x => x > 0); // true

// reduce: riduce l'array a un singolo valore
let somma = numeri.reduce((acc, curr) => acc + curr, 0); // 15
```

Oggetti: Proprietà e Metodi

Creazione e Accesso agli Oggetti

```
// Creazione oggetto letterale
let persona = {
  nome: "Mario",
  cognome: "Rossi",
  età: 30,
  indirizzo: {
    via: "Via Roma 1",
    città: "Milano"
  }
};

// Accesso alle proprietà
let nome = persona.nome;  // "Mario"
let città = persona.indirizzo.città;  // "Milano"

// Accesso alternativo (notazione con parentesi quadre)
let prop = "nome";
let valore = persona[prop];  // "Mario"
```

Modifica e Aggiunta Proprietà

```
// Modifica proprietà esistente
persona.età = 31;

// Aggiunta nuova proprietà
persona.professione = "Sviluppatore";

// Rimozione proprietà
delete persona.età;
```

Metodi negli Oggetti

```
// Aggiunta di metodi (funzioni come proprietà)
let persona = {
  nome: "Mario",
  cognome: "Rossi",
  // Metodo classico
  saluta: function() {
    return "Ciao, sono " + this.nome;
  },
  // Sintassi abbreviata (ES6)
  presentati() {
    return `Mi chiamo ${this.nome} ${this.cognome}`;
  }
};

// Chiamata di un metodo
console.log(persona.saluta());  // "Ciao, sono Mario"
```

Iterazione sugli Oggetti

```
// Ottenere tutte le chiavi
let chiavi = Object.keys(persona);  // ["nome", "cognome", ...]

// Ottenere tutti i valori
let valori = Object.values(persona);  // ["Mario", "Rossi", ...]

// Ottenere coppie chiave-valore
let entries = Object.entries(persona);
// [["nome", "Mario"], ["cognome", "Rossi"], ...]

// Iterazione con for...in
for (let key in persona) {
  console.log(key + ": " + persona[key]);
}
```

Funzionalità Avanzate

Spread e Rest con Oggetti

```
// Copia di oggetti (shallow copy)
let copia = {...persona};

// Unione di oggetti
let dettagli = {professione:
"Sviluppatore", hobby: "Lettura"};
let personaCompleta = {...persona,
...dettagli};
```

Destrutturazione

```
// Estrazione di proprietà
let {nome, cognome, età = 25} =
persona;
// nome = "Mario", cognome = "Rossi",
età = 30 (o 25 se undefined)

// Rinomina durante destrutturazione
let {nome: firstName, cognome:
lastName} = persona;
// firstName = "Mario", lastName =
"Rossi"
```

Object.assign

```
// Unione di oggetti
let personaEstesa = Object.assign({},
persona, {
  hobby: "Calcio",
  sposato: false
});

// Equivalente a:
// {...persona, hobby: "Calcio",
sposato: false}
```

Esercitazione: Rubrica Telefonica

```
// Creiamo una semplice rubrica con ricerca
let rubrica = {
 contatti: [
 {nome: "Mario", cognome: "Rossi", telefono: "3331234567"},
 {nome: "Luigi", cognome: "Verdi", telefono: "3339876543"},
 {nome: "Anna", cognome: "Bianchi", telefono: "3335554433"}
 ],

 // Metodo per aggiungere contatto
 aggiungiContatto(nome, cognome, telefono) {
 this.contatti.push({nome, cognome, telefono});
 return this.contatti.length;
 },

 // Metodo per cercare contatto
 cercaContatto(query) {
 query = query.toLowerCase();
 return this.contatti.filter(contatto =>
 contatto.nome.toLowerCase().includes(query) ||
 contatto.cognome.toLowerCase().includes(query)
 );
 }
};

// Uso della rubrica
rubrica.aggiungiContatto("Paolo", "Neri", "3337778899");
console.log(rubrica.cercaContatto("ross")); // Trova Mario Rossi
```

Controllo di Flusso: If, Switch e Cicli

Istruzioni Condizionali

If, Else If, Else

```
let età = 19;

if (età < 18) {
  console.log("Minorenne");
} else if (età === 18) {
  console.log("Appena maggiorenne");
} else {
  console.log("Maggiorenne");
}

// Forma compatta per istruzioni semplici
if (età >= 18) console.log("Maggiorenne");
```

Operatore Ternario

```
let stato = età >= 18 ? "Maggiorenne" : "Minorenne";
```

Switch/Case

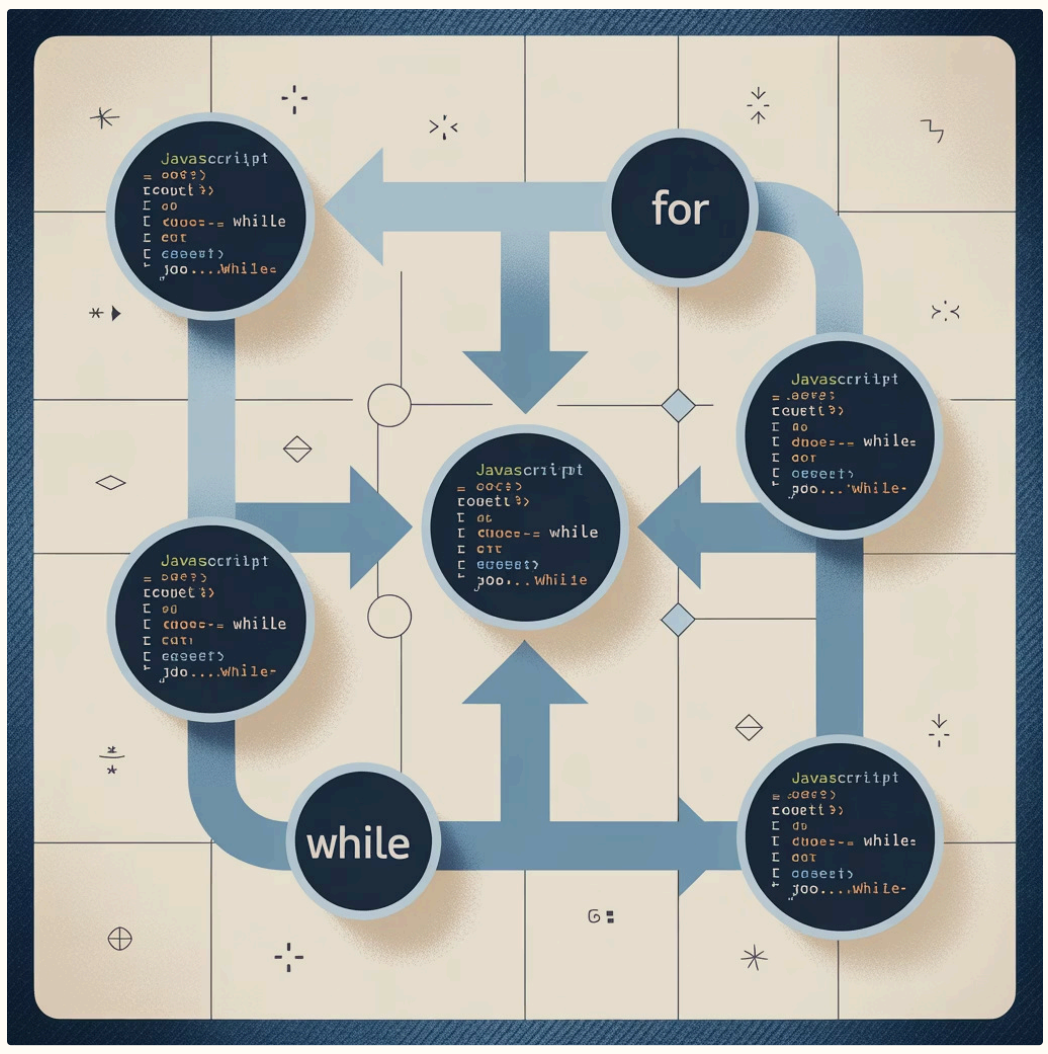
```
let giorno = 2;
let nomeGiorno;

switch (giorno) {
  case 1:
    nomeGiorno = "Lunedì";
    break;
  case 2:
    nomeGiorno = "Martedì";
    break;
  case 3:
    nomeGiorno = "Mercoledì";
    break;
  // ...altri casi...
  default:
    nomeGiorno = "Giorno non valido";
}

console.log(nomeGiorno);  // "Martedì"
```

Ricorda sempre di usare il `break` per evitare il "fall-through" involontario.

Cicli



Ciclo For

```
// Ciclo for standard
for (let i = 0; i < 5; i++) {
  console.log(`Iterazione ${i}`);
}

// Ciclo for con array
let frutta = ["mela", "pera", "banana"];
for (let i = 0; i < frutta.length; i++) {
  console.log(frutta[i]);
}

// for...of (per iterabili come array)
for (let elemento of frutta) {
  console.log(elemento);
}

// for...in (per oggetti, itera sulle chiavi)
let persona = {nome: "Mario", età: 30};
for (let chiave in persona) {
  console.log(`${chiave}: ${persona[chiave]}`);
}
```

Cicli While e Do...While

```
// While: verifica condizione prima di eseguire
let contatore = 0;
while (contatore < 5) {
  console.log(contatore);
  contatore++;
}

// Do...While: esegue almeno una volta
let numero = 10;
do {
  console.log(numero);
  numero--;
} while (numero > 0);
```

Controllo dei Cicli

Break

Interrompe l'esecuzione di un ciclo:

```
for (let i = 0; i < 10; i++) {
  if (i === 5) {
    break;  // Esce dal ciclo quando i = 5
  }
  console.log(i);  // Stampa solo 0, 1, 2, 3, 4
}
```

Continue

Salta all'iterazione successiva:

```
for (let i = 0; i < 10; i++) {
  if (i % 2 === 0) {
    continue;  // Salta i numeri pari
  }
  console.log(i);  // Stampa solo 1, 3, 5, 7, 9
}
```

Cicli Annidati

Cicli all'interno di altri cicli:

```
// Tabellina
for (let i = 1; i <= 5; i++) {
  for (let j = 1; j <= 5; j++) {
  console.log(`${i} x ${j} = ${i * j}`);
  }
  console.log('-----------');
}
```

Funzioni: Dichiarazione e Utilizzo

Definizione delle Funzioni

Dichiarazione di Funzione

```
// Sintassi classica
function somma(a, b) {
  return a + b;
}

// Chiamata
let risultato = somma(5, 3);  // 8
```

Le funzioni dichiarate sono soggette a hoisting (possono essere chiamate prima della dichiarazione nel codice).

Espressione di Funzione

```
// Funzione anonima assegnata a variabile
let moltiplica = function(a, b) {
  return a * b;
};

// Chiamata
let prodotto = moltiplica(4, 2);  // 8
```

Le funzioni espresse non sono soggette a hoisting (devono essere definite prima di essere chiamate).

Arrow Function (ES6)

```
// Sintassi concisa (=>)
let dividi = (a, b) => a / b;

// Con più istruzioni
let potenza = (base, esponente) => {
  let risultato = 1;
  for (let i = 0; i < esponente; i++) {
    risultato *= base;
  }
  return risultato;
};
```

Le arrow function hanno un comportamento diverso per quanto riguarda il `this` e non possono essere usate come costruttori.

Parametri delle Funzioni

```
// Parametri di default (ES6)
function saluta(nome = "Utente") {
  return `Ciao ${nome}!`;
}
saluta();  // "Ciao Utente!"
saluta("Mario");  // "Ciao Mario!"

// Rest parameters (...) per numero variabile di argomenti
function sommaNumeri(...numeri) {
  return numeri.reduce((totale, numero) => totale + numero, 0);
}
sommaNumeri(1, 2, 3, 4);  // 10

// Destrutturazione nei parametri
function stampaDatiPersona({nome, età}) {
  console.log(`${nome} ha ${età} anni`);
}
stampaDatiPersona({nome: "Mario", età: 30, professione: "Sviluppatore"});  // "Mario ha 30 anni"
```

Closures e Scope delle Funzioni

Una closure è una funzione che "ricorda" il suo ambiente lessicale, anche quando viene eseguita al di fuori di esso:

```
function creaContatore() {
  let conteggio = 0;  // Variabile privata

  return function() {
    conteggio++;  // Accede alla variabile del contesto esterno
    return conteggio;
  };
}

let contatore = creaContatore();
console.log(contatore());  // 1
console.log(contatore());  // 2
console.log(contatore());  // 3

// La variabile "conteggio" non è accessibile dall'esterno
// ma la funzione ritornata la "ricorda" attraverso la closure
```

Le closure sono utili per creare dati privati e funzioni factory personalizzate.

Avanzato: Callback, Arrow Functions e Scope

Arrow Functions: Sintassi ed Esempi

Le arrow functions offrono una sintassi concisa e un comportamento speciale di this:

```
// Forma base
let somma = (a, b) => a + b;

// Con un solo parametro (parentesi opzionali)
let quadrato = x => x * x;

// Senza parametri (parentesi obbligatorie)
let saluta = () => "Ciao!";

// Con blocco di codice
let potenza = (base, exp) => {
  let risultato = 1;
  for (let i = 0; i < exp; i++) {
    risultato *= base;
  }
  return risultato;
};

// Return di un oggetto (parentesi obbligatorie)
let creaPunto = (x, y) => ({ x: x, y: y });
```

Il Comportamento di this

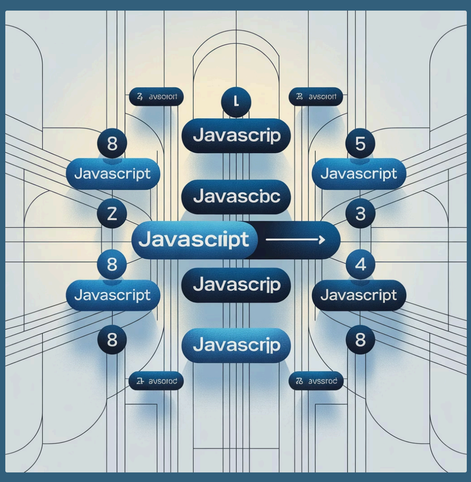
```
// Nelle funzioni tradizionali, 'this' dipende dal contesto di
chiamata
function Persona() {
  this.età = 0;

  // 'this' si riferisce all'oggetto timer, non a Persona
  setInterval(function() {
    this.età++;  // NON funziona come previsto
  }, 1000);
}

// Con arrow function, 'this' è legato lessicalmente
function PersonaCorretta() {
  this.età = 0;

  // 'this' si riferisce ancora a PersonaCorretta
  setInterval(() => {
    this.età++;  // Funziona come previsto
  }, 1000);
}
```

Callback Functions



Le callback sono funzioni passate come argomenti ad altre funzioni, per essere eseguite più tardi:

```
// Funzione che accetta una callback
function elaboraDati(dati, callback) {
  // Elaborazione dati...
  console.log("Elaborazione dati in corso...");

  // Chiamata della callback con il risultato
  callback("Risultato: " + dati);
}

// Utilizzo della funzione con una callback
elaboraDati("input utente", function(risultato) {
  console.log(risultato);
});

// Stessa cosa con arrow function
elaboraDati("input utente", risultato => {
  console.log(risultato);
});
```

Higher Order Functions

Funzioni che accettano altre funzioni come argomenti o restituiscono funzioni:

```
// Funzione che restituisce una funzione
function moltiplicatorePer(fattore) {
  // Restituisce una funzione personalizzata
  return numero => numero * fattore;
}

// Creazione di funzioni specializzate
const doppio = moltiplicatorePer(2);
const triplo = moltiplicatorePer(3);

console.log(doppio(5));  // 10
console.log(triplo(5));  // 15
```

Scope Lessicale e Chiusure (Closures)

Lo scope lessicale significa che una funzione utilizza le variabili dal contesto in cui è definita, non da dove è chiamata:

```
function esterna() {
  const messaggio = "Sono nella funzione esterna";

  function interna() {
    console.log(messaggio);  // Accede alla variabile della funzione esterna
  }

  return interna;  // Restituisce la funzione interna
}

const funzioneChiusura = esterna();  // Ottiene la funzione interna
funzioneChiusura();  // "Sono nella funzione esterna"
```

Esempi pratici di closures:

Contatore Privato

```
function creaContatore() {
  let conteggio = 0;  // Variabile privata

  return {
    incrementa: () => ++conteggio,
    decrementa: () => --conteggio,
    valore: () => conteggio
  };
}

const contatore = creaContatore();
console.log(contatore.incrementa());  // 1
console.log(contatore.incrementa());  // 2
console.log(contatore.valore());      // 2
console.log(contatore.decrementa());  // 1

// conteggio è inaccessibile dall'esterno
```

Factory di Funzioni

```
function creaFiltro(proprietà, valore) {
  return oggetto => oggetto[proprietà] === valore;
}

const prodotti = [
  {nome: "Laptop", prezzo: 1000, categoria: "Elettronica"},
  {nome: "Scarpe", prezzo: 80, categoria: "Abbigliamento"},
  {nome: "Monitor", prezzo: 300, categoria: "Elettronica"}
];

const filtraElettronici = creaFiltro("categoria", "Elettronica");
const soloElettronici = prodotti.filter(filtraElettronici);
console.log(soloElettronici); // [{nome: "Laptop"...}, {nome:
"Monitor"...}]
```

Error Handling: Try, Catch, Finally

Gestione degli Errori con Try/Catch

JavaScript fornisce il costrutto try/catch/finally per gestire le eccezioni e prevenire arresti imprevisti del programma:

```
try {
  // Codice che potrebbe generare un errore
  let risultato = 10 / 0;  // Operazione valida in JS (risulta Infinity)
  console.log(risultato);

  // Generiamo un errore
  let obj = null;
  console.log(obj.proprietà);  // Errore! Accesso a proprietà di null
} catch (errore) {
  // Gestione dell'errore
  console.error("Si è verificato un errore:", errore.message);
  // "Si è verificato un errore: Cannot read properties of null"
} finally {
  // Codice che viene sempre eseguito, indipendentemente dagli errori
  console.log("Operazioni concluse, con o senza errori");
}

console.log("Il programma continua...");  // L'esecuzione prosegue
```

Lanciare Errori Personalizzati

È possibile generare errori intenzionalmente usando l'istruzione `throw`:

```
function dividi(a, b) {
  if (b === 0) {
    throw new Error("Impossibile dividere per zero!");
  }

  return a / b;
}

try {
  let risultato = dividi(10, 0);
  console.log(risultato);
} catch (errore) {
  console.error("Errore nella divisione:", errore.message);
  // "Errore nella divisione: Impossibile dividere per zero!"
}
```

Tipi di Errori in JavaScript

Error

La classe base per tutti gli errori.

```
throw new Error("Messaggio generico");
```

SyntaxError

Errori di sintassi nel codice.

```
// Generato automaticamente
// quando c'è un errore di sintassi
function() { // Manca il nome
```

TypeError

Quando un valore non è del tipo atteso.

```
let x = null;
x.toString(); // TypeError
```

ReferenceError

Riferimento a variabile non dichiarata.

```
console.log(variabileInesistente);
```

RangeError

Valore numerico fuori dall'intervallo.

```
let arr = new Array(-1); // RangeError
```

Errori Personalizzati

Estendendo la classe Error.

```
class ValidationError extends Error {
  constructor(message) {
    super(message);
    this.name = "ValidationError";
  }
}

throw new ValidationError("Dati non validi");
```

Esempio: Validazione Input Utente

```
function validaEmail(email) {
  if (!email) {
  throw new Error("L'email non può essere vuota");
  }

  const regexEmail = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  if (!regexEmail.test(email)) {
  throw new Error("Formato email non valido");
  }

  return true;
}

function registraUtente(nome, email) {
  try {
  validaEmail(email);
  // Prosegue con la registrazione se l'email è valida
  console.log(`Utente ${nome} registrato con successo con email ${email}`);
  return true;
  } catch (errore) {
  console.error(`Errore durante la registrazione: ${errore.message}`);
  return false;
  }
}

// Test
registraUtente("Mario", "mario@example.com"); // Successo
registraUtente("Luigi", "luigi@invalid"); // Errore
```

Introduzione alla Programmazione Orientata agli Oggetti

Concetto di Oggetto e Prototipo

In JavaScript, l'OOP è basata sui prototipi anziché sulle classi. Ogni oggetto ha un prototipo che funge da "modello" e permette l'ereditarietà:

```
// Oggetto letterale di base
let persona = {
  nome: "Mario",
  saluta() {
    return `Ciao, sono ${this.nome}`;
  }
};

// Accesso al prototipo
console.log(Object.getPrototypeOf(persona));  // Object.prototype
```

Constructor Function

Metodo tradizionale per creare "classi" in JavaScript:

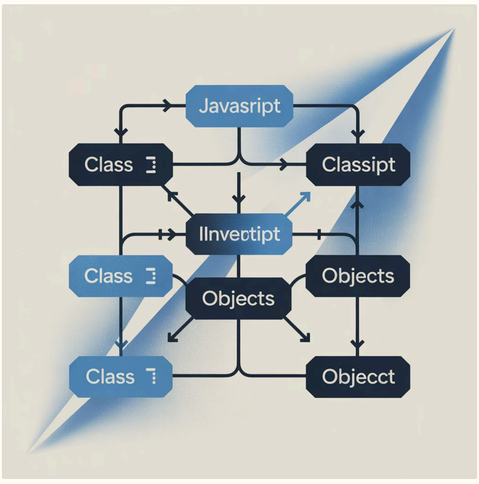
```
// Constructor function
function Persona(nome, età) {
  this.nome = nome;
  this.età = età;
}

// Aggiunta di metodi al prototipo
Persona.prototype.saluta = function() {
  return `Ciao, sono ${this.nome} e ho ${this.età} anni`;
};

// Creazione di istanze
let mario = new Persona("Mario", 30);
let luigi = new Persona("Luigi", 28);

console.log(mario.saluta());  // "Ciao, sono Mario e ho 30 anni"
```

Classi ES6



ES6 ha introdotto una sintassi basata su classi (che sono comunque basate sui prototipi internamente):

```
// Dichiarazione di classe
class Persona {
  // Costruttore
  constructor(nome, età) {
    this.nome = nome;
    this.età = età;
  }

  // Metodi di istanza
  saluta() {
    return `Ciao, sono ${this.nome}`;
  }

  // Metodi statici (chiamati sulla classe, non sull'istanza)
  static creaPersonaAnonima() {
    return new Persona("Anonimo", 0);
  }
}

// Uso della classe
const mario = new Persona("Mario", 30);
console.log(mario.saluta());  // "Ciao, sono Mario"

const anonimo = Persona.creaPersonaAnonima();
console.log(anonimo.nome);  // "Anonimo"
```

Ereditarietà e Override dei Metodi

1

Ereditarietà con Prototype

```
// Constructor function di base
function Animale(nome) {
  this.nome = nome;
}
Animale.prototype.verso = function() {
  return "Verso generico";
};

// Constructor function derivata
function Cane(nome, razza) {
  // Chiamata al costruttore padre
  Animale.call(this, nome);
  this.razza = razza;
}

// Impostazione dell'ereditarietà
Cane.prototype = Object.create(Animale.prototype);
Cane.prototype.constructor = Cane;

// Override di un metodo
Cane.prototype.verso = function() {
  return "Bau bau!";
};

// Nuovo metodo specifico
Cane.prototype.scodinzola = function() {
  return `${this.nome} scodinzola felice!`;
};

const rex = new Cane("Rex", "Pastore Tedesco");
console.log(rex.verso());     // "Bau bau!"
console.log(rex.scodinzola()); // "Rex scodinzola felice!"
```

2

Ereditarietà con Classi ES6

```
// Classe base
class Veicolo {
  constructor(marca, anno) {
  this.marca = marca;
  this.anno = anno;
  }

  descrivi() {
  return `Veicolo ${this.marca} del ${this.anno}`;
  }

  avvia() {
  return "Veicolo avviato";
  }
}

// Classe derivata
class Auto extends Veicolo {
  constructor(marca, anno, modello) {
  // Chiamata al costruttore della classe padre
  super(marca, anno);
  this.modello = modello;
  }

  // Override di un metodo
  descrivi() {
  return `Auto ${this.marca} ${this.modello} del ${this.anno}`;
  }

  // Nuovo metodo specifico
  guidaAutomatica() {
  return `${this.modello} in modalità guida automatica`;
  }
}

const miaAuto = new Auto("Tesla", 2022, "Model 3");
console.log(miaAuto.descrivi()); // "Auto Tesla Model 3 del 2022"
console.log(miaAuto.avvia()); // "Veicolo avviato" (ereditato)
```

Array Avanzati: Map, Filter, Reduce, Find

Metodi Funzionali per Array

I metodi funzionali permettono di manipolare gli array in modo più dichiarativo ed elegante, senza cicli espliciti:

1

map()

Trasforma ogni elemento dell'array, creando un nuovo array di uguale lunghezza.

```
const numeri = [1, 2, 3, 4, 5];

// Raddoppia ogni numero
const doppi = numeri.map(num => num * 2);
// [2, 4, 6, 8, 10]

// Formatta come stringhe
const stringhe = numeri.map(num => `Numero: ${num}`);
// ["Numero: 1", "Numero: 2", ...]
```

2

filter()

Seleziona gli elementi che soddisfano una condizione.

```
const numeri = [1, 2, 3, 4, 5, 6, 7, 8];

// Filtro dei numeri pari
const pari = numeri.filter(num => num % 2 === 0);
// [2, 4, 6, 8]

// Filtro dei numeri maggiori di 5
const grandi = numeri.filter(num => num > 5);
// [6, 7, 8]
```

3

reduce()

Riduce l'array a un singolo valore, accumulando i risultati.

```
const numeri = [1, 2, 3, 4, 5];

// Somma di tutti gli elementi
const somma = numeri.reduce(
  (accumulatore, valore) => accumulatore + valore,
  0  // valore iniziale
);
// 15

// Trovare il valore massimo
const massimo = numeri.reduce(
  (max, valore) => valore > max ? valore : max,
  numeri[0]
);
// 5
```

4

find() e findIndex()

Trovano il primo elemento che soddisfa una condizione.

```
const utenti = [
  { id: 1, nome: "Mario", attivo: false },
  { id: 2, nome: "Luigi", attivo: true },
  { id: 3, nome: "Anna", attivo: true }
];

// Trova il primo utente attivo
const utenteAttivo = utenti.find(utente => utente.attivo);
// { id: 2, nome: "Luigi", attivo: true }

// Trova l'indice del primo utente attivo
const indice = utenti.findIndex(utente => utente.attivo);
// 1
```

Esempio: Elaborare una Lista di Prodotti

```
// Lista di prodotti di un e-commerce
const prodotti = [
  { id: 1, nome: "Laptop", prezzo: 1200, categoria: "Elettronica", disponibile: true },
  { id: 2, nome: "Smartphone", prezzo: 800, categoria: "Elettronica", disponibile: true },
  { id: 3, nome: "Libro JavaScript", prezzo: 35, categoria: "Libri", disponibile: true },
  { id: 4, nome: "Monitor 4K", prezzo: 350, categoria: "Elettronica", disponibile: false },
  { id: 5, nome: "Tastiera", prezzo: 80, categoria: "Accessori", disponibile: true }
];

// 1. Filtra solo i prodotti disponibili
const disponibili = prodotti.filter(p => p.disponibile);

// 2. Estrai solo prodotti elettronici disponibili
const elettronicaDisponibile = prodotti
  .filter(p => p.categoria === "Elettronica" && p.disponibile);

// 3. Calcola il valore totale del carrello
const valoreTotale = prodotti
  .filter(p => p.disponibile)
  .reduce((totale, p) => totale + p.prezzo, 0);

// 4. Formatta i prodotti per la visualizzazione
const prodottiFormattati = prodotti.map(p => ({
  nome: p.nome,
  prezzo: `€${p.prezzo.toFixed(2)}`,
  stato: p.disponibile ? "Disponibile" : "Esaurito"
}));

// 5. Trova un prodotto specifico
const laptop = prodotti.find(p => p.nome === "Laptop");

console.log("Valore totale:", valoreTotale);  // 2115
console.log("Prodotti formattati:", prodottiFormattati);
```

Concatenare Metodi Array

È possibile concatenare più metodi in sequenza per operazioni complesse:

```
// Calcola la somma dei quadrati dei numeri pari
const risultato = [1, 2, 3, 4, 5, 6]
  .filter(num => num % 2 === 0) // [2, 4, 6]
  .map(num => num * num) // [4, 16, 36]
  .reduce((sum, num) => sum + num, 0); // 56

console.log(risultato); // 56
```

Lavorare con le Date e le RegExp

L'Oggetto Date

JavaScript fornisce l'oggetto Date per manipolare date e orari:

```
// Creazione di date
const adesso = new Date();  // Data e ora correnti
const data1 = new Date('2023-01-15');  // Da stringa ISO
const data2 = new Date(2023, 0, 15);  // Anno, mese (0-11), giorno
const data3 = new Date(1673740800000);  // Da timestamp (millisecondi)

// Ottenere componenti della data
const anno = adesso.getFullYear();  // es. 2023
const mese = adesso.getMonth();     // 0-11 (0 = gennaio)
const giorno = adesso.getDate();    // 1-31
const ore = adesso.getHours();      // 0-23
const minuti = adesso.getMinutes(); // 0-59
const secondi = adesso.getSeconds(); // 0-59

// Impostare componenti della data
adesso.setFullYear(2024);
adesso.setMonth(6);  // Luglio
adesso.setDate(15);
```

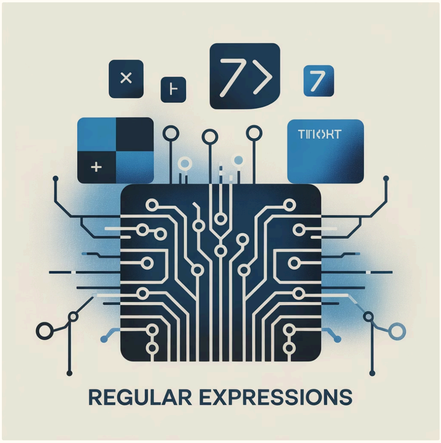
Formattazione delle Date

```
// Formati standard
const data = new Date('2023-04-15T14:30:00');
console.log(data.toString());      // Sat Apr 15 2023 14:30:00
console.log(data.toDateString());  // Sat Apr 15 2023
console.log(data.toTimeString());  // 14:30:00 GMT+0200
console.log(data.toISOString());   // 2023-04-15T12:30:00.000Z
console.log(data.toLocaleDateString('it-IT'));  // 15/04/2023
console.log(data.toLocaleTimeString('it-IT'));  // 14:30:00

// Formattazione personalizzata con Intl.DateTimeFormat
const formattatore = new Intl.DateTimeFormat('it-IT', {
  year: 'numeric',
  month: 'long',
  day: 'numeric',
  weekday: 'long',
  hour: '2-digit',
  minute: '2-digit'
});

console.log(formattatore.format(data));
// "sabato 15 aprile 2023, 14:30"
```

Espressioni Regolari (RegExp)



Le espressioni regolari sono pattern utilizzati per trovare corrispondenze in stringhe:

```
// Creazione di RegExp
const regex1 = /pattern/;  // Letterale
const regex2 = new RegExp('pattern');  // Costruttore

// Modificatori (flags)
const regexCase = /javascript/i;  // i = ignora maiuscole/minuscole
const regexGlobal = /a/g;  // g = trova tutte le occorrenze
const regexMultiline = /^inizio/m;  // m = modalità multilinea

// Test di base (verifica se il pattern esiste)
const testo = "JavaScript è fantastico!";
const contiene = /script/i.test(testo);  // true

// Trovare corrispondenze
const matches = testo.match(/a/gi);  // ['a', 'a', 'a']

// Sostituzione con RegExp
const nuovoTesto = testo.replace(/javascript/i, "TypeScript");
// "TypeScript è fantastico!"
```

Pattern RegExp Comuni

```
// Pattern per e-mail
const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
emailRegex.test("user@example.com");  // true
emailRegex.test("invalid.email");     // false

// Pattern per numeri di telefono italiani
const telRegex = /^(\+39)?[\s]?3\d{2}[\s]?\d{6,7}$/;
telRegex.test("+39 333 1234567");  // true

// Pattern per codice fiscale italiano
const cfRegex = /^[A-Z]{6}\d{2}[A-Z]\d{2}[A-Z]\d{3}[A-Z]$/i;
cfRegex.test("RSSMRA80A01H501W");  // true

// Pattern per password (min 8 caratteri, almeno 1 maiuscola,
// 1 minuscola, 1 numero)
const pwdRegex = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d).{8,}$/;
pwdRegex.test("Password123");  // true
```

Esempio: Validazione Email con RegExp

```
function validazioneEmail(email) {
 // Pattern completo per email
 const emailRegex = /^(([^<>()\[\]\\.,;:\s@"]+(\.[^<>()\[\]\\.,;:\s@"]+)*)|(".+"))@((\[[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}])|(([a-zA-Z\-0-9]+\.)+[a-zA-Z]{2,}))$/;

 // Controlla se l'email rispetta il pattern
 if (!emailRegex.test(email)) {
 return {
 valido: false,
 messaggio: "Formato email non valido"
 };
 }

 // Controlli aggiuntivi
 const [utente, dominio] = email.split('@');

 if (utente.length > 64) {
 return {
 valido: false,
 messaggio: "Nome utente troppo lungo (max 64 caratteri)"
 };
 }

 if (dominio.length > 255) {
 return {
 valido: false,
 messaggio: "Dominio troppo lungo (max 255 caratteri)"
 };
 }

 return {
 valido: true,
 messaggio: "Email valida"
 };
}

// Test della funzione
console.log(validazioneEmail("user@example.com"));
console.log(validazioneEmail("invalid.email"));
console.log(validazioneEmail("user@domain"));
```

Gestione Eventi e DOM

Selezionare Elementi nel DOM

JavaScript fornisce potenti metodi per selezionare elementi dalla pagina HTML:

```
// Selezionare elementi singoli
const titolo = document.getElementById('titolo');
const primoBottone = document.querySelector('.btn-primary');
const form = document.querySelector('form.login');

// Selezionare gruppi di elementi
const paragrafi = document.getElementsByTagName('p');
const bottoni = document.getElementsByClassName('btn');
const campiInput = document.querySelectorAll('input[type="text"]');

// Navigare nel DOM
const genitore = titolo.parentElement;
const primoFiglio = genitore.firstElementChild;
const prossimoElemento = titolo.nextElementSibling;
const elementoPrecedente = titolo.previousElementSibling;
```

Manipolare il DOM

Modificare Contenuto

```
// Cambiare il testo
titolo.textContent = "Nuovo titolo";  // Solo testo
titolo.innerHTML = "Titolo con <em>enfasi</em>";  // HTML

// Modificare attributi
const immagine = document.querySelector('img');
immagine.src = "nuova-immagine.jpg";
immagine.alt = "Descrizione alternativa";

// Verificare/ottenere attributi
const href = document.querySelector('a').getAttribute('href');
const hasClass = titolo.hasAttribute('class');

// Gestire classi CSS
titolo.classList.add('evidenziato');
titolo.classList.remove('nascosto');
titolo.classList.toggle('attivo');  // Aggiunge se assente, rimuove se presente
titolo.classList.replace('vecchiaClasse', 'nuovaClasse');
```

Creare e Inserire Elementi

```
// Creare nuovi elementi
const nuovoParagrafo = document.createElement('p');
nuovoParagrafo.textContent = "Questo è un nuovo paragrafo.";

// Aggiungere agli elementi esistenti
document.body.appendChild(nuovoParagrafo);  // In fondo
document.body.prepend(nuovoParagrafo);      // All'inizio

// Inserire in posizioni specifiche
const riferimento = document.querySelector('.riferimento');
riferimento.before(nuovoParagrafo);  // Prima del riferimento
riferimento.after(nuovoParagrafo);   // Dopo il riferimento

// Rimuovere elementi
nuovoParagrafo.remove();  // Rimuove l'elemento
// Metodo vecchio stile:
// nuovoParagrafo.parentElement.removeChild(nuovoParagrafo);
```

Modificare Stili

```
// Stili inline
titolo.style.color = "blue";
titolo.style.fontSize = "24px";
titolo.style.fontWeight = "bold";
titolo.style.backgroundColor = "#f0f0f0";

// Ottenere stili calcolati
const stili = window.getComputedStyle(titolo);
console.log(stili.color);
console.log(stili.fontSize);

// Modificare visibilità
elemento.style.display = "none";   // Nasconde
elemento.style.display = "block";  // Mostra
elemento.style.visibility = "hidden";  // Nasconde ma mantiene lo spazio
```

Gestione Eventi

```
// Metodo moderno: addEventListener
const bottone = document.querySelector('button');

bottone.addEventListener('click', function(evento) {
 console.log('Il bottone è stato cliccato!');
 console.log('Elemento cliccato:', evento.target);
 console.log('Coordinate del click:', evento.clientX, evento.clientY);

 // Prevenire il comportamento predefinito
 evento.preventDefault();

 // Fermare la propagazione dell'evento
 evento.stopPropagation();
});

// Rimuovere un event listener (serve riferimento alla stessa funzione)
function gestoreClick(e) {
 console.log('Cliccato!');
}
bottone.addEventListener('click', gestoreClick);
bottone.removeEventListener('click', gestoreClick);

// Eventi principali
document.addEventListener('DOMContentLoaded', () => {
 console.log('DOM completamente caricato');
});

window.addEventListener('load', () => {
 console.log('Pagina completamente caricata (incluse immagini)');
});

document.querySelector('input').addEventListener('keyup', (e) => {
 console.log('Tasto premuto:', e.key);
});

document.querySelector('select').addEventListener('change', (e) => {
 console.log('Valore selezionato:', e.target.value);
});

document.querySelector('form').addEventListener('submit', (e) => {
 e.preventDefault(); // Impedisce l'invio del form
 console.log('Form inviato!');
});
```

Promesse e Gestione Asincrona

Le Promise in JavaScript

Le Promise sono un pattern per gestire operazioni asincrone, rappresentando un valore che potrebbe essere disponibile ora, in futuro, o mai.

```
// Creazione di una Promise
const miaPromise = new Promise((resolve, reject) => {
  // Operazione asincrona simulata
  setTimeout(() => {
    const successo = Math.random() > 0.5;

    if (successo) {
      // Operazione completata con successo
      resolve("Operazione completata!");
    } else {
      // Operazione fallita
      reject(new Error("Qualcosa è andato storto"));
    }
  }, 2000);
});

// Utilizzo della Promise
miaPromise
  .then(risultato => {
    // Gestione del successo
    console.log("Successo:", risultato);
    return "Valore modificato";  // Valore passato al prossimo then
  })
  .then(nuovoRisultato => {
    console.log("Nuovo risultato:", nuovoRisultato);
  })
  .catch(errore => {
    // Gestione dell'errore
    console.error("Errore:", errore.message);
  })
  .finally(() => {
    // Eseguito sia in caso di successo che di errore
    console.log("Operazione conclusa");
  });
```

Combinare Promise

```
// Eseguire multiple Promise in parallelo
const promise1 = fetch('/api/users');
const promise2 = fetch('/api/products');
const promise3 = fetch('/api/orders');

// Attendere che tutte le Promise siano risolte
Promise.all([promise1, promise2, promise3])
  .then(responses => {
    // Array di tutte le risposte nell'ordine originale
    return Promise.all(responses.map(r => r.json()));
  })
  .then(data => {
    const [users, products, orders] = data;
    console.log("Utenti:", users);
    console.log("Prodotti:", products);
    console.log("Ordini:", orders);
  })
  .catch(error => {
    // Se una qualsiasi Promise fallisce
    console.error("Una delle richieste è fallita:", error);
  });

// Attendere la prima Promise che si risolve
Promise.race([
  fetch('/api/server1/data'),
  fetch('/api/server2/data'),
  fetch('/api/server3/data')
])
.then(firstResponse => firstResponse.json())
.then(data => console.log("Dati dal server più veloce:", data))
.catch(error => console.error("Errore:", error));
```

Async/Await

Introdotto in ES2017, async/await è una sintassi più leggibile per lavorare con le Promise:

Funzioni Async

```
// Definizione di una funzione async
async function fetchUserData(userId) {
  try {
    // await sospende l'esecuzione fino al completamento della Promise
    const response = await fetch(`/api/users/${userId}`);

    if (!response.ok) {
      throw new Error(`HTTP error! status: ${response.status}`);
    }

    const userData = await response.json();
    return userData;  // Ritorna una Promise risolta con userData
  } catch (error) {
    console.error("Errore durante il recupero dei dati:", error);
    throw error;  // Propaga l'errore
  }
}

// Utilizzo della funzione async
fetchUserData(123)
  .then(user => console.log("Utente:", user))
  .catch(error => console.error("Gestione errore:", error));
```

Esecuzione Parallela con Async/Await

```
async function fetchMultipleResources() {
  try {
    // Avvia le richieste in parallelo
    const userPromise = fetch('/api/users').then(r => r.json());
    const productPromise = fetch('/api/products').then(r => r.json());

    // Attende il completamento di entrambe
    const [users, products] = await Promise.all([
      userPromise,
      productPromise
    ]);

    console.log("Utenti:", users);
    console.log("Prodotti:", products);

    return { users, products };
  } catch (error) {
    console.error("Errore:", error);
    throw error;
  }
}
```

Loop con Async/Await

```
async function processItems(items) {
  const results = [];

  // Sequenziale: un item alla volta
  for (const item of items) {
    const result = await processItem(item);
    results.push(result);
  }

  // Parallelo: tutti gli item contemporaneamente
  // const promises = items.map(item => processItem(item));
  // const results = await Promise.all(promises);

  return results;
}

// Funzione di supporto
async function processItem(item) {
  // Simulazione elaborazione
  await new Promise(r => setTimeout(r, 100));
  return `Processed: ${item}`;
}
```

Esempio: Chiamata API Asincrona

```
// Funzione per caricare dati da un'API di notizie
async function caricaNotizie(categoria = 'tecnologia', pagina = 1) {
  const contenitore = document.getElementById('notizie-container');
  const loaderEl = document.getElementById('loader');

  try {
    // Mostra loader
    loaderEl.style.display = 'block';
    contenitore.innerHTML = '';

    // Effettua la richiesta API
    const response = await fetch(
      `https://api.example.com/news?category=${categoria}&page=${pagina}&limit=10`
    );

    if (!response.ok) {
      throw new Error(`Errore HTTP ${response.status}`);
    }

    // Piccolo ritardo artificiale per vedere il loader
    await new Promise(r => setTimeout(r, 800));

    // Elabora i dati
    const { articles, totalPages } = await response.json();

    if (articles.length === 0) {
      contenitore.innerHTML = '
```

Nessuna notizia trovata

`';  return;  }  // Visualizza le notizie  articles.forEach(article => {  const articleEl = document.createElement('article');  articleEl.className = 'news-item';  articleEl.innerHTML = `

${article.title}

${new Date(article.publishedAt).toLocaleDateString()}

${article.summary}

Leggi di più `;  contenitore.appendChild(articleEl);  });  // Crea paginazione  creaPaginazione(pagina, totalPages, categoria);  } catch (error) { console.error('Errore durante il caricamento delle notizie:', error);  contenitore.innerHTML = `

Si è verificato un errore durante il caricamento delle notizie.

${error.message}

Riprova

`; } finally { // Nascondi loader loaderEl.style.display = 'none'; } } // Funzione di supporto per creare la paginazione function creaPaginazione(paginaCorrente, totalePagine, categoria) { const paginationEl = document.getElementById('pagination'); paginationEl.innerHTML = ''; if (totalePagine <= 1) return; // Logica di paginazione... // (codice omesso per brevità) } // Inizializzazione document.addEventListener('DOMContentLoaded', () => { caricaNotizie(); // Event listener per i filtri delle categorie document.querySelectorAll('.category-filter').forEach(button => { button.addEventListener('click', () => { const categoria = button.dataset.category; caricaNotizie(categoria); }); }); });

Gestione Form e Validazione Input

Lavorare con i Form

I form sono elementi fondamentali per la raccolta di dati dagli utenti. JavaScript permette di controllare l'invio, leggere i valori e validare gli input:

```
// Selezionare un form e prevenire l'invio predefinito
const form = document.getElementById('registrazione-form');

form.addEventListener('submit', function(e) {
  e.preventDefault();  // Impedisce l'invio tradizionale del form

  // Accesso ai campi del form
  const username = document.getElementById('username').value;
  const email = document.getElementById('email').value;
  const password = document.getElementById('password').value;

  // Validazione base
  if (!username || !email || !password) {
    alert('Tutti i campi sono obbligatori');
    return;
  }

  // Se tutto è valido, puoi inviare i dati
  console.log('Dati form:', { username, email, password });

  // Qui potresti inviare i dati tramite fetch API
  // inviaFormAsync(username, email, password);

  // Oppure inviare il form in modo tradizionale
  // form.submit();
});
```

Lettura Valori dei Campi

Campi di Testo

```
// Input di testo, email, password, ecc.
const testo = document.getElementById('nome').value;
const email = document.querySelector('[type="email"]').value;

// Textarea
const messaggio = document.getElementById('messaggio').value;
```

Checkbox e Radio

```
// Singola checkbox
const accettaTermini = document.getElementById('termini').checked;

// Gruppo di radio button
const genere = document.querySelector('input[name="genere"]:checked').value;

// Gruppo di checkbox
const hobbies = Array.from(

document.querySelectorAll('input[name="hobby"]:checked')
).map(input => input.value);
```

Select e Multi-select

```
// Select normale
const paese = document.getElementById('paese').value;

// Select con opzioni multiple
const lingue = Array.from(

document.getElementById('lingue').selectedOptions
).map(option => option.value);

// Alternativa per select multipla
const lingueBis = Array.from(
  document.querySelectorAll('#lingue option:checked')
).map(option => option.value);
```

Validazione Input

Validazione con Attributi HTML5

HTML5 fornisce attributi per la validazione base:

```
<input type="email" required>
<input type="url" pattern="https?://.*">
<input type="number" min="1" max="100">
```

Controllo della validità tramite JavaScript:

```
// Verificare se un singolo campo è valido
const isValid = document.getElementById('email').validity.valid;

// Verificare e accedere ai dettagli della validità
const validityState = input.validity;
if (validityState.valueMissing) {
  console.log('Campo obbligatorio');
} else if (validityState.typeMismatch) {
  console.log('Formato non valido');
}

// Verificare tutto il form
const isFormValid = form.checkValidity();
```

Validazione Personalizzata

Per validazioni più complesse, è necessario scrivere codice JavaScript personalizzato:

```
function validaPassword(password) {
  // Almeno 8 caratteri, 1 maiuscola, 1 minuscola, 1 numero
  const regex = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d).{8,}$/;
  return regex.test(password);
}

function validaNomeUtente(username) {
  // Solo lettere, numeri e underscore, 3-20 caratteri
  const regex = /^[a-zA-Z0-9_]{3,20}$/;
  return regex.test(username);
}

// Controllo che due password corrispondano
function passwordCorrispondono(password1, password2) {
  return password1 === password2;
}
```

Feedback di Validazione

```
// Funzione per mostrare errori di validazione
function mostraErrore(input, messaggio) {
 // Rimuovi eventuali messaggi di errore precedenti
 const parent = input.parentElement;
 const errorElement = parent.querySelector('.error-message');
 if (errorElement) {
 errorElement.remove();
 }

 // Aggiungi la classe di errore all'input
 input.classList.add('input-error');

 // Crea e aggiungi il nuovo messaggio di errore
 const error = document.createElement('div');
 error.className = 'error-message';
 error.textContent = messaggio;
 parent.appendChild(error);
}

// Funzione per rimuovere gli errori
function rimuoviErrore(input) {
 input.classList.remove('input-error');

 const parent = input.parentElement;
 const errorElement = parent.querySelector('.error-message');
 if (errorElement) {
 errorElement.remove();
 }
}

// Validazione in tempo reale
document.getElementById('email').addEventListener('input', function() {
 const email = this.value.trim();
 const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

 if (email === '') {
 mostraErrore(this, 'L\'email è obbligatoria');
 } else if (!emailRegex.test(email)) {
 mostraErrore(this, 'Inserisci un indirizzo email valido');
 } else {
 rimuoviErrore(this);
 }
});

// Validazione password durante la digitazione
document.getElementById('password').addEventListener('input', function() {
 const password = this.value;

 // Verifica la forza della password
 let forza = 0;
 const lunghezzaOk = password.length >= 8;
 const haNumeri = /\d/.test(password);
 const haMinuscole = /[a-z]/.test(password);
 const haMaiuscole = /[A-Z]/.test(password);
 const haSpeciali = /[^a-zA-Z0-9]/.test(password);

 forza += lunghezzaOk ? 1 : 0;
 forza += haNumeri ? 1 : 0;
 forza += haMinuscole ? 1 : 0;
 forza += haMaiuscole ? 1 : 0;
 forza += haSpeciali ? 1 : 0;

 // Mostra indicatore di forza
 const forzaIndicator = document.getElementById('password-strength');
 if (forza < 2) {
 forzaIndicator.textContent = 'Debole';
 forzaIndicator.className = 'strength-weak';
 } else if (forza < 4) {
 forzaIndicator.textContent = 'Media';
 forzaIndicator.className = 'strength-medium';
 } else {
 forzaIndicator.textContent = 'Forte';
 forzaIndicator.className = 'strength-strong';
 }
});
```

Moduli e Struttura del Codice

IIFE (Immediately Invoked Function Expression)

Un pattern per creare scope isolati e prevenire la contaminazione dello scope globale:

```
// Definizione e invocazione immediata di una funzione
(function() {
  // Variabili private, non accessibili dall'esterno
  const contatore = 0;
  const API_KEY = "abc123";

  function incrementa() {
    return contatore++;
  }

  // Espone solo ciò che vogliamo all'esterno
  window.miaApp = {
    incrementa: contatore: incrementa,
    versione: "1.0.0"
  };
})();

// Utilizzo
console.log(miaApp.versione);  // "1.0.0"
miaApp.incrementaContatore();  // Incrementa il contatore
// console.log(contatore);  // Errore! Non accessibile
```

Namespace Pattern

Un pattern per organizzare il codice in oggetti annidati, riducendo i conflitti di nomi:

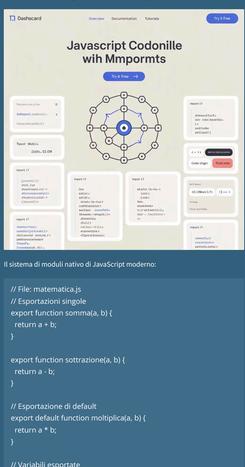
```
// Definizione del namespace
var MiaApp = MiaApp || {};

// Sottomoduli
MiaApp.utils = {
  formattaData: function(data) {
    return new Date(data).toLocaleDateString();
  },
  generaId: function() {
    return Math.random().toString(36).substr(2, 9);
  }
};

MiaApp.ui = {
  mostraAlert: function(messaggio) {
    alert(messaggio);
  },
  nascondiElemento: function(id) {
    document.getElementById(id).style.display = 'none';
  }
};

// Utilizzo
MiaApp.ui.mostraAlert('Ciao!');
const id = MiaApp.utils.generaId();
```

ES Modules (import/export)



Il sistema di moduli nativo di JavaScript moderno:

```
// File: matematica.js
// Esportazioni singole
export function somma(a, b) {
  return a + b;
}

export function sottrazione(a, b) {
  return a - b;
}

// Esportazione di default
export default function moltiplica(a, b) {
  return a * b;
}

// Variabili esportate
export const PI = 3.14159;


// File: utility.js
// Funzioni helper
function formattaNumero(n) {
  return n.toFixed(2);
}

function capitalizza(str) {
  return str.charAt(0).toUpperCase() + str.slice(1);
}

// Esportazione multipla
export { formattaNumero, capitalizza };


// File: main.js
// Importa default e membri singoli
import moltiplica, { somma, PI } from './matematica.js';

// Importa e rinomina
import { sottrazione as differenza } from './matematica.js';

// Importa tutto in un oggetto
import * as Utils from './utility.js';

// Utilizzo
console.log(somma(5, 3));        // 8
console.log(moltiplica(4, 2));   // 8
console.log(differenza(10, 4));  // 6
console.log(PI);                 // 3.14159
console.log(Utils.formattaNumero(PI));  // "3.14"
console.log(Utils.capitalizza('ciao'));  // "Ciao"
```

Best Practice per Codice Modulare

Responsabilità Singola

Ogni modulo dovrebbe avere una sola responsabilità e un solo motivo per cambiare.

- Separa la logica di business dalla UI
- Crea moduli per gestione API, validazione, utility, ecc.
- Evita "god objects" che fanno troppe cose

Incapsulamento

Nascondi i dettagli implementativi e esponi solo un'interfaccia pubblica controllata.

- Usa closure per creare variabili private
- Esporta solo ciò che è necessario all'esterno
- Usa convenzioni di nomenclatura (es. _prefisso per metodi privati)

Dipendenze Esplicite

Rendi chiare le dipendenze tra moduli anziché affidarti a variabili globali.

- Passa le dipendenze come parametri
- Importa esplicitamente ciò che ti serve
- Usa l'iniezione delle dipendenze dove appropriato

Esempio: Suddividere un Piccolo Progetto

```
// Struttura file di un'app To-Do
/*
/js
 /modules
  config.js // Configurazioni, costanti
  api.js // Comunicazione con il server/localStorage
  taskModel.js // Logica dati per le attività
  taskValidator.js // Validazione delle attività
  eventBus.js // Comunicazione tra componenti
 /components
  taskForm.js // Componente form per aggiungere/modificare
  taskList.js // Visualizzazione lista attività
  taskItem.js // Singolo elemento della lista
 app.js // Inizializzazione e coordinamento
 utils.js // Funzioni di utilità generale
index.html
style.css
*/

// Esempio di implementazione:

// api.js
export async function getTasks() {
  const tasks = JSON.parse(localStorage.getItem('tasks') || '[]');
  return tasks;
}

export async function saveTask(task) {
  const tasks = await getTasks();
  if (task.id) {
    // Aggiornamento
    const index = tasks.findIndex(t => t.id === task.id);
    if (index >= 0) tasks[index] = task;
  } else {
    // Nuovo task
    task.id = Date.now().toString();
    tasks.push(task);
  }
  localStorage.setItem('tasks', JSON.stringify(tasks));
  return task;
}

export async function deleteTask(id) {
  const tasks = await getTasks();
  const newTasks = tasks.filter(t => t.id !== id);
  localStorage.setItem('tasks', JSON.stringify(newTasks));
}

// taskValidator.js
export function validateTask(task) {
  const errors = {};

  if (!task.title.trim()) {
  errors.title = "Il titolo è obbligatorio";
  }

  if (task.title.length > 100) {
  errors.title = "Il titolo non può superare 100 caratteri";
  }

  if (task.dueDate && isNaN(new Date(task.dueDate).getTime())) {
  errors.dueDate = "Data non valida";
  }

  return {
  valid: Object.keys(errors).length === 0,
  errors
  };
}

// taskForm.js
import { saveTask } from './api.js';
import { validateTask } from './taskValidator.js';
import { emitEvent } from './eventBus.js';

export function initTaskForm() {
  const form = document.getElementById('task-form');

  form.addEventListener('submit', async (e) => {
  e.preventDefault();

  const task = {
  id: form.dataset.taskId || '',
  title: document.getElementById('task-title').value,
  description: document.getElementById('task-description').value,
  dueDate: document.getElementById('task-due-date').value,
  completed: document.getElementById('task-completed').checked
  };

  const validation = validateTask(task);
  if (!validation.valid) {
  // Mostra errori
  showValidationErrors(validation.errors);
  return;
  }

  try {
  await saveTask(task);
  form.reset();
  form.dataset.taskId = '';

  // Notifica gli altri componenti
  emitEvent('task-saved', task);
  } catch (error) {
  console.error('Error saving task:', error);
  }
  });
}

// app.js (entry point)
import { initTaskForm } from './components/taskForm.js';
import { initTaskList } from './components/taskList.js';

document.addEventListener('DOMContentLoaded', () => {
  initTaskForm();
  initTaskList();
});
```

Debugging e Ottimizzazione

Strumenti di Debugging

Console del Browser

Lo strumento più basilare ma potente per il debugging:

```
// Stampa semplice
console.log("Messaggio informativo");

// Visualizzazione strutturata
console.table([
  { nome: "Mario", età: 30 },
  { nome: "Luigi", età: 28 }
]);

// Raggruppamento di output correlati
console.group("Dati utente");
console.log("Nome: Mario");
console.log("Email: mario@example.com");
console.groupEnd();

// Misurazione del tempo
console.time("operazione");
// ...codice da misurare...
console.timeEnd("operazione");

// Visualizzazione condizionale
console.assert(1 === 2, "Questo verrà mostrato");

// Livelli di log
console.info("Informazione");
console.warn("Avviso");
console.error("Errore");
```

Breakpoints

Punti di interruzione per esaminare il codice durante l'esecuzione:

```
// Breakpoint tramite codice
debugger;  // L'esecuzione si ferma qui quando i DevTools sono aperti

// Nei DevTools:
// 1. Apri Sources/Sorgenti nel pannello DevTools
// 2. Naviga al file JavaScript desiderato
// 3. Fai clic sul numero di riga per impostare un breakpoint
// 4. Durante l'esecuzione, puoi:
//    - Vedere le variabili locali
//    - Valutare espressioni nella console
//    - Step over, step into, step out
//    - Continuare l'esecuzione
```

Try/Catch e Monitoring

Gestione degli errori e monitoraggio:

```
// Cattura errori specifici
try {
  // Codice potenzialmente problematico
  const data = JSON.parse(jsonString);
} catch (error) {
  console.error("Errore di parsing JSON:", error.message);
  // Gestione dell'errore...
}

// Monitoraggio globale degli errori
window.addEventListener('error', function(event) {
  console.error('Errore non gestito:', event.error);
  // Potresti inviare l'errore a un servizio di logging
});

// Monitoraggio delle promise non gestite
window.addEventListener('unhandledrejection', function(event) {
  console.error('Promise non gestita:', event.reason);
});
```

Identificare e Risolvere Bug Comuni

Bug di Tipizzazione

```
// Problema: comparazione di tipi diversi
if (userId == "123") {  // == converte i tipi
  // ...
}

// Soluzione: usa l'uguaglianza stretta
if (userId === "123") {  // Controlla valore e tipo
  // ...
}

// Problema: NaN
let risultato = "ciao" / 2;  // NaN
if (risultato == NaN) {  // Non funziona mai!
  // ...
}

// Soluzione:
if (isNaN(risultato)) {
  // ...
}
```

Scope e Hoisting

```
// Problema: variabile non definita nel punto giusto
function esempio() {
  console.log(x);  // undefined (hoisting)
  var x = 5;
  console.log(x);  // 5
}

// Problema: let e const non sono hoisted allo stesso modo
function esempioBug() {
  console.log(y);  // Error: Cannot access 'y' before initialization
  let y = 5;
}

// Soluzione: dichiara le variabili prima dell'uso
function esempioCorretto() {
  let y;
  console.log(y);  // undefined
  y = 5;
  console.log(y);  // 5
}
```

Problemi con il this

```
// Problema: perdita del contesto this
const user = {
  name: "Mario",
  greet: function() {
    console.log(`Ciao, sono ${this.name}`);
  }
};

// Funziona
user.greet();  // "Ciao, sono Mario"

// Problema - this si riferisce alla window o undefined (strict mode)
const greetFn = user.greet;
greetFn();  // "Ciao, sono undefined"

// Soluzioni:
// 1. Bind
const boundGreet = user.greet.bind(user);
boundGreet();  // "Ciao, sono Mario"

// 2. Arrow function che "cattura" this
const user2 = {
  name: "Luigi",
  greet: function() {
    // this qui è user2
    const arrowFn = () => {
      // this qui è ancora user2
      console.log(`Ciao, sono ${this.name}`);
    };
    return arrowFn;
  }
};
const greetFn2 = user2.greet();
greetFn2();  // "Ciao, sono Luigi"
```

Problemi Asincroni

```
// Problema: accesso a dati prima che siano pronti
function caricaDati() {
  let dati;
  fetch('/api/data')
    .then(response => response.json())
    .then(result => {
      dati = result;
    });
  return dati;  // Sempre undefined!
}

// Soluzione: usa Promise o async/await
async function caricaDatiCorretto() {
  const response = await fetch('/api/data');
  const dati = await response.json();
  return dati;  // Corretto
}
```

Analisi delle Performance Script

```
// Misurazione del tempo di esecuzione
function misuraTempo(callback) {
 const inizioTempo = performance.now();
 callback();
 const fineTempo = performance.now();
 return fineTempo - inizioTempo;
}

// Esempio d'uso
const tempoA = misuraTempo(() => {
 // Algoritmo A...
 for (let i = 0; i < 1000000; i++) {
 Math.sqrt(i);
 }
});

const tempoB = misuraTempo(() => {
 // Algoritmo B...
 const numeri = [];
 for (let i = 0; i < 1000000; i++) {
 numeri.push(i);
 }
});

console.log(`Algoritmo A: ${tempoA.toFixed(2)}ms`);
console.log(`Algoritmo B: ${tempoB.toFixed(2)}ms`);

// Performance API più avanzata
performance.mark('inizio-elaborazione');
// ... codice da misurare ...
performance.mark('fine-elaborazione');
performance.measure('tempo-elaborazione', 'inizio-elaborazione', 'fine-elaborazione');

const misure = performance.getEntriesByType('measure');
console.log(`Tempo di elaborazione: ${misure[0].duration.toFixed(2)}ms`);

// Profiling con DevTools:
// 1. Apri il pannello Performance nei DevTools
// 2. Fai clic su Record
// 3. Esegui le operazioni da analizzare
// 4. Fai clic su Stop
// 5. Analizza il flame chart, il riepilogo delle chiamate e le timeline
```

Sicurezza in JavaScript

Rischi di Sicurezza Comuni

Cross-Site Scripting (XSS)

Quando un attaccante riesce a iniettare codice JavaScript dannoso che viene eseguito nella pagina:

```
// Vulnerabile a XSS
function mostraMessaggio(messaggio) {
  // PERICOLOSO: inserimento diretto di input utente nel DOM

  document.getElementById('output').innerHTML = messaggio;
}

// Se l'utente inserisce:
// <script>alert('Hacked!');</script>
// o
// <img src="x" onerror="rubaDati()">
// Il codice verrà eseguito nel browser della vittima
```

Cross-Site Request Forgery (CSRF)

Quando un sito malevolo fa eseguire azioni non autorizzate su un sito in cui l'utente è autenticato:

```
// Vulnerabile a CSRF
// Se l'utente è autenticato su example.com, un altro sito può fare:
<img src="https://example.com/api/trasferisci?destinatario=hacker&importo=1000">

// La richiesta includerà automaticamente i cookie di autenticazione
// e potrebbe eseguire operazioni non autorizzate
```

Injection di Codice

Quando l'input dell'utente viene valutato come codice:

```
// Vulnerabile a code injection
function calcolaRisultato(espressione) {
  // PERICOLOSO: valutazione diretta dell'input
  return eval(espressione);
}

// Un utente malintenzionato potrebbe inserire:
// "alert('Hacked!')" o
// "fetch('/api/segreti').then(r=>r.json()).then(inviaAlServer)"
```

Best Practice per la Sicurezza

Sanitizzazione Input e Output

```
// Sanitizzazione dell'output per prevenire XSS
function mostraMessaggioSicuro(messaggio) {
  // Converte i caratteri speciali in entità HTML
  const sanitizzato = messaggio
    .replace(/&/g, "&")
    .replace(/</g, ">")
    .replace(/"/g, """)
    .replace(/'/g, "");

  document.getElementById('output').innerHTML = sanitizzato;
}

// Meglio ancora: usa textContent invece di innerHTML
function mostraMessaggioSicuro2(messaggio) {
  document.getElementById('output').textContent = messaggio;
}

// Per contenuto HTML più complesso, usa librerie come DOMPurify
// import DOMPurify from 'dompurify';
// const pulito = DOMPurify.sanitize(inputHtml);
// elemento.innerHTML = pulito;
```

Evitare eval e Funzioni Pericolose

```
// Alternative sicure a eval
// Invece di:
eval('const x = 10; console.log(x);');

// Usa:
// Per espressioni matematiche semplici
function calcolaEspressione(espressione) {
  // Verifica che contenga solo caratteri permessi
  if (!/^[0-9+\-*/().]+$/.test(espressione)) {
    throw new Error("Espressione non valida");
  }

  // Usa Function invece di eval (leggermente più sicuro)
  return Function('"use strict"; return (${espressione})')();
}

// Per JSON, usa sempre JSON.parse invece di eval
// Invece di:
const dati = eval('(' + jsonString + ')');

// Usa:
const dati = JSON.parse(jsonString);
```

Protezione dai CSRF e Principi di Sicurezza

```
// Prevenzione CSRF con token
// 1. Genera un token lato server e includilo nel form
const form = document.querySelector('form');
const csrfToken = document.querySelector('meta[name="csrf-token"]').content;

// 2. Aggiungi il token a ogni richiesta fetch
async function inviaRichiestaSicura(url, dati) {
  const response = await fetch(url, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'X-CSRF-Token': csrfToken  // Token anti-CSRF
    },
    body: JSON.stringify(dati),
    credentials: 'same-origin'  // Includi sempre i cookie
  });

  return response.json();
}

// Same-Origin Policy e CORS
// Il browser impedisce alle pagine di fare richieste a domini diversi
// a meno che il server non lo permetta esplicitamente con header CORS

// Evita l'esecuzione automatica di codice da fonti non attendibili
// Invece di:
const script = document.createElement('script');
script.src = 'https://sito-non-sicuro.com/script.js';
document.head.appendChild(script);

// Valuta sempre la provenienza degli script esterni
// Usa Subresource Integrity quando possibile:
//
```

Validazione e Sanitizzazione Input

Un approccio completo per gestire l'input utente in modo sicuro:

```
// Classe utility per validazione e sanitizzazione
class Validatore {
  // Validazione email
  static isEmailValido(email) {
    return /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email);
  }

  // Validazione per input numerico
  static isNumeroValido(numero) {
    return !isNaN(parseFloat(numero)) && isFinite(numero);
  }

  // Sanitizzazione per testo da mostrare in HTML
  static sanitizzaHTML(testo) {
    if (!testo) return '';
    return testo
      .replace(/&/g, "&")
      .replace(/</g, ">")
      .replace(/"/g, """)
      .replace(/'/g, "");
  }

  // Sanitizzazione per URL
  static sanitizzaURL(url) {
    try {
      const urlObj = new URL(url);
      // Permetti solo http e https
      if (urlObj.protocol !== 'http:' && urlObj.protocol !== 'https:') {
        return '';
      }
      return urlObj.toString();
    } catch (e) {
      return '';
    }
  }

  // Validazione per username (alfanumerico + underscore)
  static isUsernameValido(username) {
    return /^[a-zA-Z0-9_]{3,20}$/.test(username);
  }

  // Sanitizzazione per input che verrà usato in query SQL
  // (applicabile lato server, ma utile come esempio)
  static sanitizzaSQL(input) {
    // In realtà dovresti usare prepared statements o ORM
    // Questa è solo una dimostrazione base
    if (!input) return '';
    return input
      .replace(/'/g, "''")
      .replace(/\\/g, '\\\\');
  }
}

// Esempio d'uso
function gestisciFormContatto() {
  const email = document.getElementById('email').value.trim();
  const messaggio = document.getElementById('messaggio').value.trim();

  // Validazione
  if (!Validatore.isEmailValido(email)) {
    mostraErrore('email', 'Inserisci un indirizzo email valido');
    return false;
  }

  if (messaggio.length < 10 || messaggio.length > 1000) {
    mostraErrore('messaggio', 'Il messaggio deve essere tra 10 e 1000 caratteri');
    return false;
  }

  // Sanitizzazione prima di mostrare
  const anteprima = `
```

Email: ${Validatore.sanitizzaHTML(email)}

Messaggio: ${Validatore.sanitizzaHTML(messaggio)}

`;document.getElementById('anteprima').innerHTML = anteprima; // Invio sicuro dei dati inviaRichiestaSicura('/api/contatto', { email, messaggio }); return true; }

Tipologie di Pattern e Design Pattern base

Cos'è un Design Pattern?

I design pattern sono soluzioni consolidate a problemi ricorrenti nella progettazione software. Forniscono un vocabolario comune e approcci standardizzati che migliorano la manutenibilità e leggibilità del codice.

Pattern Creazionali

1

Singleton

Garantisce che una classe abbia una sola istanza e fornisce un punto di accesso globale ad essa.

```
// Implementazione del Singleton
const Database = (function() {
  let instance;

  // Costruttore privato
  function createInstance() {
    const connessione = {
      host: "localhost",
      connect() {
        console.log("Connesso al DB");
        return this;
      },
      query(sql) {
        console.log(`Esecuzione query: ${sql}`);
        return [/* risultati */];
      }
    };
    return connessione;
  }

  return {
    // Metodo pubblico per ottenere l'istanza
    getInstance: function() {
      if (!instance) {
        instance = createInstance();
      }
      return instance;
    }
  };
})();

// Utilizzo
const db1 = Database.getInstance().connect();
const db2 = Database.getInstance();

console.log(db1 === db2);  // true, è la stessa istanza
```

2

Factory

Fornisce un'interfaccia per creare oggetti senza specificare le classi concrete.

```
// Implementazione del Factory Pattern
// Definizione delle "classi" prodotto
function SedanAuto(options) {
  this.colore = options.colore || "nero";
  this.porte = 4;
  this.tipo = "sedan";
}

function SUVAuto(options) {
  this.colore = options.colore || "bianco";
  this.porte = 5;
  this.tipo = "suv";
  this.trazione = "integrale";
}

function SportivaAuto(options) {
  this.colore = options.colore || "rosso";
  this.porte = 2;
  this.tipo = "sportiva";
  this.velMax = 300;
}

// Factory per creare diversi tipi di auto
const AutoFactory = {
  creaAuto(tipo, options = {}) {
    switch (tipo.toLowerCase()) {
      case "sedan":
        return new SedanAuto(options);
      case "suv":
        return new SUVAuto(options);
      case "sportiva":
        return new SportivaAuto(options);
      default:
        throw new Error(`Tipo di auto non supportato: ${tipo}`);
    }
  }
};

// Utilizzo
const miaSedan = AutoFactory.creaAuto("sedan", { colore: "blu" });
const mioSUV = AutoFactory.creaAuto("suv");

console.log(miaSedan);  // SedanAuto { colore: "blu", porte: 4, tipo: "sedan" }
console.log(mioSUV);    // SUVAuto { colore: "bianco", porte: 5, ... }
```

3

Builder

Separa la costruzione di un oggetto complesso dalla sua rappresentazione, permettendo di creare diverse configurazioni.

```
// Implementazione del Builder Pattern
class FormBuilder {
  constructor() {
    this.form = {};
  }

  withDimension(width, height) {
    this.form.width = width;
    this.form.height = height;
    return this;
  }

  withTitle(title) {
    this.form.title = title;
    return this;
  }

  withField(name, type, options = {}) {
    if (!this.form.fields) this.form.fields = [];
    this.form.fields.push({
      name,
      type,
      required: options.required || false,
      label: options.label || name,
      ...options
    });
    return this;
  }

  withSubmitButton(text = "Invia") {
    this.form.submitButton = { text };
    return this;
  }

  build() {
    // Validazione
    if (!this.form.fields || !this.form.fields.length === 0) {
      throw new Error("Il form deve avere almeno un campo");
    }
    if (!this.form.submitButton) {
      this.withSubmitButton();
    }

    return this.form;
  }
}

// Utilizzo
const loginForm = new FormBuilder()
  .withTitle("Accesso Utente")
  .withDimension(400, 300)
  .withField("email", "email", { required: true, label: "Indirizzo Email" })
  .withField("password", "password", { required: true, minLength: 8 })
  .withSubmitButton("Accedi")
  .build();

console.log(loginForm);
```

Pattern Comportamentali

1

Observer

Definisce una dipendenza uno-a-molti tra oggetti, cosicché quando un oggetto cambia stato, tutti i suoi dipendenti vengono notificati e aggiornati automaticamente.

```
// Implementazione dell'Observer Pattern
class Subject {
  constructor() {
    this.observers = [];
  }

  subscribe(observer) {
    this.observers.push(observer);
  }

  unsubscribe(observerToRemove) {
    this.observers = this.observers.filter(
      observer => observer !== observerToRemove
    );
  }

  notify(data) {
    this.observers.forEach(observer => observer.update(data));
  }
}

class Observer {
  constructor(name) {
    this.name = name;
  }

  update(data) {
    console.log(`${this.name} ha ricevuto l'aggiornamento:`, data);
  }
}

// Utilizzo
const newsLetter = new Subject();

const user1 = new Observer("Mario");
const user2 = new Observer("Luigi");
const user3 = new Observer("Anna");

newsLetter.subscribe(user1);
newsLetter.subscribe(user2);
newsLetter.subscribe(user3);

newsLetter.notify({ titolo: "Nuova funzionalità disponibile!" });

// Mario ha ricevuto l'aggiornamento: { titolo: "Nuova funzionalità..." }
// Luigi ha ricevuto l'aggiornamento: { titolo: "Nuova funzionalità..." }
// Anna ha ricevuto l'aggiornamento: { titolo: "Nuova funzionalità..." }

newsLetter.unsubscribe(user2);
newsLetter.notify({ titolo: "Aggiornamento di sicurezza" });

// Solo Mario e Anna riceveranno questo aggiornamento
```

2

Strategy

Permette di definire una famiglia di algoritmi, incapsularli e renderli intercambiabili.

```
// Implementazione del Strategy Pattern
// Definizione delle strategie
const paymentStrategies = {
  creditCard: {
    pay(amount) {
      console.log(`Pagamento di €${amount} con carta di credito`);
      // Logica per processare il pagamento con carta
      return true;
    }
  },

  paypal: {
    pay(amount) {
      console.log(`Pagamento di €${amount} con PayPal`);
      // Logica per processare il pagamento con PayPal
      return true;
    }
  },

  bankTransfer: {
    pay(amount) {
      console.log(`Pagamento di €${amount} con bonifico bancario`);
      // Logica per processare il bonifico
      return true;
    }
  }
};

// Contesto che usa la strategia
class ShoppingCart {
  constructor() {
    this.items = [];
    this.paymentStrategy = null;
  }

  addItem(item) {
    this.items.push(item);
  }

  calculateTotal() {
    return this.items.reduce((total, item) => total + item.price * item.quantity, 0);
  }

  setPaymentStrategy(strategyKey) {
    if (!paymentStrategies[strategyKey]) {
      throw new Error(`Strategia di pagamento non supportata: ${strategyKey}`);
    }
    this.paymentStrategy = paymentStrategies[strategyKey];
  }

  checkout() {
    if (!this.paymentStrategy) {
      throw new Error("Seleziona un metodo di pagamento");
    }

    const amount = this.calculateTotal();
    return this.paymentStrategy.pay(amount);
  }
}

// Utilizzo
const cart = new ShoppingCart();
cart.addItem({ name: "Laptop", price: 999, quantity: 1 });
cart.addItem({ name: "Mouse", price: 25.99, quantity: 2 });

// Cliente sceglie il metodo di pagamento
cart.setPaymentStrategy("paypal");
cart.checkout();  // Pagamento di €1050.98 con PayPal

// Cliente cambia idea sul metodo di pagamento
cart.setPaymentStrategy("creditCard");
cart.checkout();  // Pagamento di €1050.98 con carta di credito
```

Esempio Pratico: Implementazione Pattern Observer

```
// Implementazione completa di un sistema di eventi basato su Observer

class EventEmitter {
  constructor() {
    this.events = {};
  }

  // Iscriviti a un evento
  on(eventName, callback) {
    if (!this.events[eventName]) {
      this.events[eventName] = [];
    }
    this.events[eventName].push(callback);

    // Restituisce una funzione per disiscriversi
    return () => this.off(eventName, callback);
  }

  // Iscriviti a un evento solo una volta
  once(eventName, callback) {
    const unsubscribe = this.on(eventName, (...args) => {
      unsubscribe();
      callback(...args);
    });
    return unsubscribe;
  }

  // Disiscriviti da un evento
  off(eventName, callback) {
    if (!this.events[eventName]) return;

    this.events[eventName] = this.events[eventName].filter(cb => cb !== callback);
  }

  // Emetti un evento con dati
  emit(eventName, ...args) {
    if (!this.events[eventName]) return;

    this.events[eventName].forEach(callback => {
      try {
        callback(...args);
      } catch (error) {
        console.error(`Errore nell'handler di ${eventName}:`, error);
      }
    });
  }
}

// Esempio d'uso: Sistema di notifiche in un'app
class NotificationSystem extends EventEmitter {
  constructor() {
    super();
    this.notifications = [];
  }

  addNotification(notification) {
    this.notifications.push({
      id: Date.now(),
      read: false,
      timestamp: new Date(),
      ...notification
    });

    this.emit('new-notification', notification);

    if (this.notifications.filter(n => !n.read).length >= 5) {
      this.emit('many-unread', this.notifications.filter(n => !n.read));
    }
  }

  markAsRead(id) {
    const notification = this.notifications.find(n => n.id === id);
    if (notification) {
      notification.read = true;
      this.emit('notification-read', notification);
    }
  }

  markAllAsRead() {
    this.notifications.forEach(n => n.read = true);
    this.emit('all-read');
  }
}

// Utilizzo
const notifications = new NotificationSystem();

// Componente UI che ascolta le notifiche
const notificationBadge = {
  count: 0,

  init() {
    // Iscriviti agli eventi
    notifications.on('new-notification', () => {
      this.count++;
      this.updateUI();
    });

    notifications.on('notification-read', () => {
      this.count = Math.max(0, this.count - 1);
      this.updateUI();
    });

    notifications.on('all-read', () => {
      this.count = 0;
      this.updateUI();
    });

    // Avviso solo al superamento di 5 notifiche non lette
    notifications.on('many-unread', (unreadNotifications) => {
      console.log(`Hai ${unreadNotifications.length} notifiche non lette!`);
    });
  },

  updateUI() {
    console.log(`Badge aggiornato: ${this.count} notifiche non lette`);
    // In un'app reale, aggiorna il DOM
  }
};

// Inizializzazione
notificationBadge.init();

// Simulazione arrivo notifiche
notifications.addNotification({ title: "Nuovo messaggio", message: "Ciao, come stai?" });
notifications.addNotification({ title: "Promemoria", message: "Meeting alle 15:00" });
notifications.markAsRead(notifications.notifications[0].id);
notifications.addNotification({ title: "Nuova email", message: "Newsletter settimanale" });
```

Progetti Guidati: Calcolatrice Web

Sviluppo di una Calcolatrice Web

In questo progetto guidato, realizzeremo una calcolatrice web funzionale utilizzando HTML, CSS e JavaScript. La calcolatrice supporterà le operazioni di base, gestirà gli input dell'utente e mostrerà i risultati in tempo reale.

Struttura HTML

```
<!-- index.html -->
<!DOCTYPE html>
<html lang="it">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Calcolatrice JavaScript</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <div class="calculator">
    <div class="display">
      <div id="previous-operand"></div>
      <div id="current-operand"></div>
    </div>

    <div class="buttons">
      <button class="operator" data-action="clear">AC</button>
      <button class="operator" data-action="delete">DEL</button>
      <button class="operator" data-action="percent">%</button>
      <button class="operator" data-action="divide">÷</button>

      <button class="number">7</button>
      <button class="number">8</button>
      <button class="number">9</button>
      <button class="operator" data-action="multiply">×</button>

      <button class="number">4</button>
      <button class="number">5</button>
      <button class="number">6</button>
      <button class="operator" data-action="subtract">−</button>

      <button class="number">1</button>
      <button class="number">2</button>
      <button class="number">3</button>
      <button class="operator" data-action="add">+</button>

      <button class="number">0</button>
      <button class="number">.</button>
      <button class="operator" data-action="equals" id="equals">=</button>
    </div>
  </div>

  <script src="calculator.js"></script>
</body>
</html>
```

Stile CSS

```
/* styles.css */
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
  font-family: 'Arial', sans-serif;
}

body {
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
  background-color: #f5f5f5;
}

.calculator {
  background-color: #222;
  border-radius: 10px;
  box-shadow: 0 5px 30px rgba(0, 0, 0, 0.3);
  max-width: 400px;
  width: 100%;
  overflow: hidden;
}

.display {
  background-color: #333;
  color: white;
  text-align: right;
  padding: 20px;
  font-size: 2rem;
}

#previous-operand {
  color: rgba(255, 255, 255, 0.75);
  font-size: 1.2rem;
  min-height: 24px;
}

#current-operand {
  min-height: 40px;
  word-wrap: break-word;
  word-break: break-all;
}

.buttons {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 1px;
  background-color: #555;
}

button {
  border: none;
  outline: none;
  background-color: #444;
  color: white;
  font-size: 1.5rem;
  padding: 20px;
  cursor: pointer;
  transition: background-color 0.2s;
}

button:hover {
  background-color: #555;
}

.operator {
  background-color: #f69906;
}

.operator:hover {
  background-color: #da8504;
}

#equals {
  grid-column: span 2;
}
```

Logica JavaScript

```
// calculator.js
class Calculator {
  constructor(previousOperandElement, currentOperandElement) {
    this.previousOperandElement = previousOperandElement;
    this.currentOperandElement = currentOperandElement;
    this.clear();
  }

  clear() {
    this.currentOperand = '0';
    this.previousOperand = '';
    this.operation = undefined;
    this.shouldResetScreen = false;
  }

  delete() {
    if (this.currentOperand === '0') return;
    if (this.currentOperand.length === 1) {
      this.currentOperand = '0';
    } else {
      this.currentOperand = this.currentOperand.slice(0, -1);
    }
  }

  appendNumber(number) {
    if (this.shouldResetScreen) {
      this.currentOperand = '';
      this.shouldResetScreen = false;
    }

    // Impedisce più punti decimali
    if (number === '.' && this.currentOperand.includes('.')) return;

    // Sostituisce lo 0 iniziale a meno che non sia un punto decimale
    if (this.currentOperand === '0' && number !== '.') {
      this.currentOperand = number;
    } else {
      this.currentOperand += number;
    }
  }

  chooseOperation(operation) {
    if (this.currentOperand === '') return;

    if (this.previousOperand !== '') {
      this.compute();
    }

    this.operation = operation;
    this.previousOperand = this.currentOperand;
    this.shouldResetScreen = true;
  }

  compute() {
    let computation;
    const prev = parseFloat(this.previousOperand);
    const current = parseFloat(this.currentOperand);

    if (isNaN(prev) || isNaN(current)) return;

    switch (this.operation) {
      case '+':
        computation = prev + current;
        break;
      case '-':
        computation = prev - current;
        break;
      case '×':
        computation = prev * current;
        break;
      case '÷':
        if (current === 0) {
          this.currentOperand = 'Errore';
          this.previousOperand = '';
          this.operation = undefined;
          return;
        }
        computation = prev / current;
        break;
      case '%':
        computation = prev * (current / 100);
        break;
      default:
        return;
    }

    // Arrotonda per evitare errori di precisione
    this.currentOperand = this.roundResult(computation);
    this.operation = undefined;
    this.previousOperand = '';
    this.shouldResetScreen = true;
  }

  roundResult(number) {
    // Controlla se è un numero intero
    if (Number.isInteger(number)) {
      return number.toString();
    }

    // Altrimenti formatta con un massimo di 10 decimali
    return number.toFixed(10).replace(/\.?0+$/, '');
  }

  updateDisplay() {
    this.currentOperandElement.textContent = this.currentOperand;

    if (this.operation != null) {
      this.previousOperandElement.textContent =
        `${this.previousOperand} ${this.operation}`;
    } else {
      this.previousOperandElement.textContent = '';
    }
  }
}

handleKeyboardInput() {
  // Gestisce l'input da tastiera
  if (e.key >= '0' && e.key <= '9') this.appendNumber(e.key);
  if (e.key === '.') this.appendNumber(e.key);
  if (e.key === '+' || e.key === '-' || e.key === '*' || e.key === '/') {
    if (e.key === 'Backspace') this.delete();
    if (e.key === 'Escape') this.clear();
    if (e.key === '+') this.chooseOperation('+');
    if (e.key === '-') this.chooseOperation('-');
    if (e.key === '*') this.chooseOperation('×');
    if (e.key === '/') this.chooseOperation('÷');
    if (e.key === '%') this.chooseOperation('%');
  }
}

// Inizializzazione della calcolatrice quando il DOM è pronto
document.addEventListener('DOMContentLoaded', () => {
  const previousOperandElement = document.getElementById('previous-operand');
  const currentOperandElement = document.getElementById('current-operand');
  const calculator = new Calculator(previousOperandElement, currentOperandElement);

  // Event listeners per i bottoni numerici
  document.querySelectorAll('.number').forEach(button => {
    button.addEventListener('click', () => {
      calculator.appendNumber(button.getAttribute('data-number'));
      calculator.updateDisplay();
    });
  });

  // Event listeners per i bottoni operatori
  document.querySelectorAll('.operator').forEach(button => {
    button.addEventListener('click', () => {
      const action = button.getAttribute('data-action');

      switch (action) {
        case 'add':
          calculator.chooseOperation('+');
          break;
        case 'subtract':
          calculator.chooseOperation('-');
          break;
        case 'multiply':
          calculator.chooseOperation('×');
          break;
        case 'divide':
          calculator.chooseOperation('÷');
          break;
        case 'percent':
          calculator.chooseOperation('%');
          break;
        case 'equals':
          calculator.compute();
          break;
        case 'clear':
          calculator.clear();
          break;
        case 'delete':
          calculator.delete();
          break;
      }

      calculator.updateDisplay();
    });
  });

  // Event listener per input da tastiera
  window.addEventListener('keydown', e => {
    calculator.handleKeyboardInput(e);
    calculator.updateDisplay();
  });
});
```

Funzionalità Implementate

Operazioni Base

- Addizione, sottrazione, moltiplicazione, divisione
- Calcolo percentuale
- Cancellazione completa (AC) e parziale (DEL)
- Supporto per numeri decimali

Gestione Input

- Aggiornamento in tempo reale del display
- Prevenzione input invalidi (es. multipli punti decimali)
- Supporto input da tastiera
- Visualizzazione operazione in corso

Funzionalità Aggiuntive

- Gestione degli errori (es. divisione per zero)
- Arrotondamento per evitare errori di precisione
- Reset automatico del display dopo operazioni
- Interfaccia responsive per tutti i dispositivi

Questa calcolatrice è un ottimo progetto pratico per consolidare le competenze di JavaScript. Può espandersi aggiungendo funzionalità più avanzate come operazioni scientifiche, cronologia dei calcoli, o temi personalizzabili.

Progetti Guidati: ToDo List con Storage

Sviluppo di un'App ToDo List

In questo progetto, creeremo un'applicazione completa per gestire una lista di attività (ToDo List). L'app permetterà di aggiungere, modificare, completare ed eliminare attività, con salvataggio persistente utilizzando localStorage.

Struttura HTML4

```
<!-- index.html -->
<!DOCTYPE html>
<html lang="it">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>ToDo List Application</title>
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/5.15.4/css/all.min.css">
    <link rel="stylesheet" href="styles.css">
</head>
<body>
    <div class="container">
        <header>
            <h1>ToDo List</h1>
            <div class="filters">
                <button class="filter-btn active" data-filter="all">Tutti</button>
                <button class="filter-btn" data-filter="active">Attivi</button>
                <button class="filter-btn" data-filter="completed">Completati</button>
            </div>
        </header>

        <form id="todo-form">
            <input type="text" id="todo-input" placeholder="Aggiungi un nuovo task..." autofocus>
            <button type="submit"><i class="fas fa-plus"></i></button>
        </form>

        <div class="todo-container">
            <ul id="todo-list">
                <!-- Gli elementi della lista verranno aggiunti dinamicamente -->
            </ul>
        </div>

        <div class="stats">
            <div id="items-left">0 elementi rimasti</div>
            <button id="clear-completed">Cancella completati</button>
        </div>
    </div>

    <script src="app.js"></script>
</body>
</html>
```

Stile CSS

```
/* styles.css */
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
    font-family: 'Arial', sans-serif;
}

body {
    background-color: #f5f6fa;
    color: #333;
    display: flex;
    justify-content: center;
    padding: 40px 20px;
    min-height: 100vh;
}

.container {
    max-width: 600px;
    width: 100%;
}

header {
    margin-bottom: 20px;
    text-align: center;
}

header h1 {
    font-size: 2.5rem;
    margin-bottom: 15px;
    color: #3498db;
}

.filters {
    display: flex;
    justify-content: center;
    gap: 10px;
    margin-bottom: 20px;
}

.filter-btn {
    background: none;
    border: 1px solid #ddd;
    border-radius: 4px;
    padding: 5px 10px;
    cursor: pointer;
    font-size: 0.9rem;
    transition: all 0.3s ease;
}

.filter-btn.active {
    background-color: #3498db;
    color: white;
    border-color: #3498db;
}

#todo-form {
    display: flex;
    margin-bottom: 20px;
}

#todo-input {
    flex: 1;
    padding: 12px 15px;
    border: 1px solid #ddd;
    border-radius: 4px 0 0 4px;
    font-size: 1rem;
}

#todo-form button {
    background-color: #3498db;
    color: white;
    border: none;
    border-radius: 0 4px 4px 0;
    padding: 0 15px;
    cursor: pointer;
    transition: background-color 0.3s ease;
}

#todo-form button:hover {
    background-color: #2980b9;
}

.todo-container {
    background-color: white;
    border-radius: 4px;
    box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
    overflow: hidden;
    margin-bottom: 20px;
}

#todo-list {
    list-style: none;
}

.todo-item {
    padding: 15px;
    border-bottom: 1px solid #eee;
    display: flex;
    align-items: center;
    transition: background-color 0.3s ease;
}

.todo-item:last-child {
    border-bottom: none;
}

.todo-item.hover {
    background-color: #f5f5f5;
}

.todo-item.myvi[type="checkbox"] {
    margin-right: 15px;
    transform: scale(1.2);
}

.todo-item label {
    flex: 1;
    font-size: 1.1rem;
    word-break: break-word;
}

.todo-item.completed label {
    text-decoration: line-through;
    color: #888;
}

.todo-item .actions {
    display: flex;
    gap: 5px;
}

.todo-item button {
    background: none;
    border: none;
    font-size: 1rem;
    color: #888;
    cursor: pointer;
    transition: color 0.3s ease;
    padding: 5px;
}

.todo-item button.hover {
    color: #333;
}

.todo-item .delete-btn.hover {
    color: #e74c3c;
}

.info {
    display: flex;
    justify-content: space-between;
    align-items: center;
    font-size: 0.9rem;
    color: #666;
}

#clear-completed {
    background: none;
    border: none;
    color: #e74c3c;
    cursor: pointer;
    font-size: 0.9rem;
}

#clear-completed.hover {
    text-decoration: underline;
}

/* Stile per l'input di modifica */
.edit-input {
    flex: 1;
    padding: 5px 10px;
    border: 1px solid #3498db;
    border-radius: 4px;
    font-size: 1rem;
}

/* Animazioni */
.todo-item.removed {
    animation: fadeOut 0.3s forwards;
}

@keyframes fadeOut {
    to {
        opacity: 0;
        height: 0;
        padding: 0;
        margin: 0;
        border: 0;
    }
}
```

Logica JavaScript

```
// app.js
document.addEventListener('DOMContentLoaded', () => {
    // Selettori elementi DOM
    const todoForm = document.getElementById('todo-form');
    const todoInput = document.getElementById('todo-input');
    const todoList = document.getElementById('todo-list');
    const itemsLeft = document.getElementById('items-left');
    const clearCompletedBtn = document.getElementById('clear-completed');
    const filterBtns = document.querySelectorAll('.filter-btn');

    // Stato dell'applicazione
    let todos = [];
    let filter = 'all';

    // Carica i todo dal localStorage
    function loadTodos() {
        const storedTodos = localStorage.getItem('todos');
        if (storedTodos) {
            todos = JSON.parse(storedTodos);
            renderTodos();
        }
    }

    // Salva i todo nel localStorage
    function saveTodos() {
        localStorage.setItem('todos', JSON.stringify(todos));
    }

    // Aggiorna il contatore degli elementi rimasti
    function updateItemsLeft() {
        const activeTodos = todos.filter(todo => !todo.completed);
        itemsLeft.textContent = `${activeTodos.length} elementi${activeTodos.length === 1 ? 'o' : 'i'} rimast${activeTodos.length === 1 ? 'o' : 'i'}`;
    }

    // Renderizza i todo in base al filtro corrente
    function renderTodos() {
        todoList.innerHTML = '';

        let filteredTodos = todos;
        if (filter === 'active') {
            filteredTodos = todos.filter(todo => !todo.completed);
        } else if (filter === 'completed') {
            filteredTodos = todos.filter(todo => todo.completed);
        }

        filteredTodos.forEach(todo => {
            const todoItem = document.createElement('li');
            todoItem.classList.add('todo-item');
            if (todo.completed) {
                todoItem.classList.add('completed');
            }
            todoItem.dataset.id = todo.id;

            // Crea la struttura dell'elemento todo
            todoItem.innerHTML = `
                <input type="checkbox" ${todo.completed ? 'checked' : ''}>
                <label>${todo.text}</label>
                <div class="actions">
                    <button class="edit-btn"><i class="fas fa-edit"></i></button>
                    <button class="delete-btn"><i class="fas fa-trash"></i></button>
                </div>
            `;

            todoList.appendChild(todoItem);
        });

        updateItemsLeft();
    }

    // Aggiunge un nuovo todo
    function addTodo(text) {
        const newTodo = {
            id: Date.now(),
            text,
            completed: false
        };

        todos.push(newTodo);
        saveTodos();
        renderTodos();
    }

    // Modifica un todo esistente
    function editTodo(id, newText) {
        todos = todos.map(todo => {
            if (todo.id === id) {
                return { ...todo, text: newText };
            }
            return todo;
        });

        saveTodos();
        renderTodos();
    }

    // Elimina un todo
    function deleteTodo(id) {
        todos = todos.filter(todo => todo.id !== id);
        saveTodos();
        renderTodos();
    }

    // Cambia lo stato di completamento di un todo
    function toggleTodo(id) {
        todos = todos.map(todo => {
            if (todo.id === id) {
                return { ...todo, completed: !todo.completed };
            }
            return todo;
        });

        saveTodos();
        renderTodos();
    }

    // Cancella tutti i todo completati
    function clearCompleted() {
        todos = todos.filter(todo => !todo.completed);
        saveTodos();
        renderTodos();
    }

    // Event Listeners
    // Form per aggiungere nuovi todo
    todoForm.addEventListener('submit', e => {
        e.preventDefault();
        const todoText = todoInput.value.trim();
        if (todoText !== '') {
            addTodo(todoText);
            todoInput.value = '';
        }
    });

    // Delegazione per gli eventi sulla lista dei todo
    todoList.addEventListener('click', e => {
        const todoItem = e.target.closest('.todo-item');
        if (!todoItem) return;

        const id = parseInt(todoItem.dataset.id);

        // Toggle completamento
        if (e.target.type === 'checkbox') {
            toggleTodo(id);
        }

        // Modifica
        if (e.target.closest('.edit-btn')) {
            const label = todoItem.querySelector('label');
            const currentText = label.textContent;

            // Crea input per la modifica
            const editInput = document.createElement('input');
            editInput.type = 'text';
            editInput.value = currentText;
            editInput.classList.add('edit-input');

            // Sostituisci la label con l'input
            label.replaceWith(editInput);
            editInput.focus();

            // Salva le modifiche quando l'input perde il focus
            editInput.addEventListener('blur', () => {
                const newText = editInput.value.trim();
                if (newText && newText !== currentText) {
                    editTodo(id, newText);
                } else {
                    renderTodos(); // Ripristina la label
                }
            });

            // Permetti anche di premere Enter per salvare
            editInput.addEventListener('keypress', e => {
                if (e.key === 'Enter') {
                    editInput.blur();
                }
            });
        }

        // Eliminazione
        if (e.target.closest('.delete-btn')) {
            todoItem.classList.add('removed');
            setTimeout(() => {
                deleteTodo(id);
            }, 300); // Attendi che l'animazione finisca
        }
    });

    // Filtri
    filterBtns.forEach(btn => {
        btn.addEventListener('click', () => {
            filterBtns.forEach(b => b.classList.remove('active'));
            btn.classList.add('active');
            filter = btn.dataset.filter;
            renderTodos();
        });
    });

    // Cancella completati
    clearCompletedBtn.addEventListener('click', clearCompleted);

    // Avvia l'applicazione
    loadTodos();
});
```

Logica JavaScript

Implementeremo ora la logica dell'applicazione per gestire le attività:

```
function checkOverdueTasks() {
    const today = new Date();

    todos.forEach(todo => {
        if (todo.dueDate) {
            const dueDate = new Date(todo.dueDate);

            if (
                dueDate < today &&
                !todo.completed &&
                !todo.isOverdue
            ) {
                todo.isOverdue = true;
            }
        }
    });

    renderTodos();
}
```

Date di Scadenza

Implementiamo date di scadenza per le attività con avvisi per quelle in ritardo.

Sincronizzazione Cloud

Aggiungere la sincronizzazione con un backend per accedere alla todo list da più dispositivi.

```
async function syncWithServer() {
    try {
        // Get server data
        const response = await
fetch('/api/todos', {
            method: 'GET',
            headers: {
                'Authorization': 'Bearer
${userToken}'
            }
        });

        if (response.ok) throw new
Error('Sync failed');

        const serverTodos = await
response.json();

        // Merge logic here...

        // Update UI
        renderTodos();
    } catch (error) {
        console.error('Sync error:', error);
        showNotification('Sincronizzazione
fallita');
    }
}
```

Questo progetto fornisce una solida base per comprendere la gestione dello stato, l'interazione utente e la persistenza dei dati in JavaScript. Può essere ulteriormente migliorato aggiungendo funzionalità come il drag & drop per riordinare le attività, notifiche, o addirittura convertirlo in un'applicazione più robusta utilizzando un framework come React o Vue.

Categorie e Tag

Aggiungere la possibilità di categorizzare le attività e filtrarle per categoria/tag.

```
// Esempio di struttura dati estesa
const todo = {
  id: 123456,
  text: 'Completare il progetto',
  completed: false,
  category: 'Lavoro',
  tags: ['urgente', 'javascript'],
  dueDate: '2023-12-31'
};
```

Gestione Attività

- Aggiunta di nuove attività
- Modifica delle attività esistenti
- Contrassegno come completate/attive
- Eliminazione singola attività
- Eliminazione di tutte le attività completate

Filtri e Visualizzazione

- Filtro per tutte le attività
- Filtro per attività attive
- Filtro per attività completate
- Conteggio attività rimanenti
- Animazioni per migliorare l'esperienza utente

Persistenza Dati

- Salvataggio automatico su localStorage
- Caricamento dati all'avvio dell'applicazione
- Mantenimento dello stato tra le sessioni

Interazione Utente

- Input validato (trimming delle spazio-bianco)
- Focus automatico per rapida aggiunta task
- Modifica in place con supporto tastiera
- Feedback visivo (animazioni)
- Design responsive

Possibili Estensioni

Risorse Utili e Consigli Finali

Libri e Documentazione Consigliati

Libri per Principianti

- **JavaScript: The Good Parts** di Douglas Crockford - Un classico che evidenzia le migliori caratteristiche del linguaggio
- **Eloquent JavaScript** di Marijn Haverbeke - Introduzione completa con esercizi pratici (disponibile anche online gratuitamente)
- **JavaScript: Guida completa per lo sviluppatore** di David Flanagan - Guida approfondita e completa in italiano
- **JavaScript Novice to Ninja** di Darren Jones - Approccio pratico con numerosi esempi

Libri per Livello Intermedio/Avanzato

- **You Don't Know JS** di Kyle Simpson - Serie di libri che approfondiscono i meccanismi interni di JavaScript
- **JavaScript Patterns** di Stoyan Stefanov - Raccolta di pattern e best practice
- **Secrets of the JavaScript Ninja** di John Resig e Bear Bibeault - Tecniche avanzate dai creatori di jQuery
- **Clean Code in JavaScript** di James Padolsey - Principi di codice pulito applicati a JavaScript

Documentazione Online

- **MDN Web Docs** - La documentazione di riferimento più completa e aggiornata su JavaScript
- **JavaScript.info** - Tutorial moderno con spiegazioni dettagliate (disponibile in inglese)
- **W3Schools** - Ottima risorsa per principianti con esempi semplici e interattivi
- **DevDocs.io** - Documentazione di riferimento rapida e consultabile offline
- **ECMAScript Specifications** - Specifiche ufficiali del linguaggio per approfondimenti tecnici

Risorse in Italiano

- **HTML.it** - Guide e tutorial in italiano su JavaScript e sviluppo web
- **MrWebMaster** - Tutorial di programmazione JavaScript in italiano
- **Guida JavaScript di Mozilla** - Traduzione italiana di parte della documentazione MDN
- **JavaScript Facile** - Corso gratuito online per principianti

Comunità e Forum

Community Internazionali

- **Stack Overflow** - La più grande community di domande e risposte per programmatori
- **Reddit - r/javascript** - Subreddit dedicato a discussioni su JavaScript
- **DEV Community** - Piattaforma per sviluppatori con molti articoli su JavaScript
- **freeCodeCamp Forum** - Community amichevole per chi sta imparando a programmare
- **GitHub** - Esplora progetti open source e impara dalle implementazioni reali

Community Italiane

- **Italian Coders** - Community italiana di sviluppatori con canale Discord attivo
- **Forum HTML.it** - Forum italiano con sezioni dedicate a JavaScript
- **Web Design Italia** - Gruppo Facebook dedicato al web design e allo sviluppo
- **WordPress Italia** - Anche se incentrato su WordPress, ha discussioni utili su JavaScript
- **ReactJS Italia** - Gruppo Telegram dedicato a React e JavaScript moderno

Eventi e Conferenze

- **JSDay Italia** - Conferenza italiana dedicata a JavaScript
- **CodeMotion** - Eventi di tecnologia in varie città italiane
- **Frontend Dev Conf** - Conferenza dedicata allo sviluppo frontend
- **Meetup locali** - Cerca gruppi JavaScript nella tua città su Meetup.com
- **Hackathon** - Eventi di programmazione intensiva per mettere in pratica le tue competenze

Strumenti e Risorse Online

Ambienti di Sviluppo Online

- **CodePen** - Perfetto per esperimenti rapidi con HTML, CSS e JavaScript
- **JSFiddle** - Ambiente di test con supporto per librerie esterne
- **CodeSandbox** - IDE online completo, ideale per progetti più complessi
- **Replit** - Ambiente di sviluppo collaborativo con supporto per Node.js

Corsi e Esercizi

- **freeCodeCamp** - Percorso completo e gratuito con certificazioni
- **Codecademy** - Corsi interattivi per imparare JavaScript
- **JavaScript30** - 30 giorni di sfide in JavaScript puro
- **Exercism** - Esercizi con mentoring da parte della community
- **HackerRank** - Sfide di programmazione per migliorare le tue competenze

Strumenti di Sviluppo

- **ESLint** - Analizzatore di codice per identificare problemi
- **Prettier** - Formattatore di codice per mantenere uno stile coerente
- **Webpack** - Bundler per gestire moduli e asset
- **Babel** - Compilatore per utilizzare funzionalità JavaScript moderne
- **Jest** - Framework per il testing di codice JavaScript
- **Chrome DevTools** - Strumenti di sviluppo integrati nel browser

Consigli per Continuare a Imparare

Ecco alcuni suggerimenti per continuare il tuo percorso di apprendimento di JavaScript:

1

Pratica Costante

Dedica del tempo ogni giorno alla programmazione. Anche solo 30 minuti di codice quotidiano porteranno a progressi significativi nel lungo periodo. Crea piccoli progetti personali che risolvano problemi reali.

2

Studia il Codice degli Altri

Leggere il codice di progetti open source ben scritti è un ottimo modo per imparare tecniche e pattern. Non limitarti a copiare soluzioni, ma cerca di capire il ragionamento dietro le scelte implementative.

3

Insegna agli Altri

Spiegare concetti ad altre persone è uno dei modi migliori per consolidare la tua comprensione. Considera di scrivere articoli, partecipare a forum, o aiutare altri sviluppatori alle prime armi.

4

Costruisci un Portfolio

Crea progetti personali che mostrino le tue competenze e pubblicali su GitHub. Un portfolio di progetti reali vale più di qualsiasi certificazione e dimostra la tua capacità di applicare le conoscenze.

5

Resta Aggiornato

JavaScript evolve rapidamente. Segui blog, podcast e newsletter del settore per rimanere al passo con le novità. Partecipa a eventi e conferenze quando possibile per connetterti con altri sviluppatori.

Ricorda che imparare a programmare è un viaggio continuo, non una destinazione. Anche gli sviluppatori più esperti continuano ad imparare nuove tecniche e approcci. Sii paziente con te stesso, celebra i piccoli progressi e mantieni viva la curiosità!

Buon coding! 🚀